

A red Mercedes-Benz 220 SE car is parked in a restaurant. The car is the central focus, with its rear view showing the Mercedes-Benz logo and '220 SE' badge. The restaurant has a warm, dimly lit atmosphere with brick walls and hanging pendant lights. Several people are seated at tables in the background, eating and talking. The car's license plate area is empty, and the text 'Daniel Saad' is overlaid on it.

LLM Agents

Daniel Saad

AI GENERATED

Chat Oberfläche

Analysemodell: Anthropic Claude

Sprache: Deutsch

Guide

Beginnen Sie mit einigen Beispielen

Wahl 2025

Welche Positionen hat die CDU zum Thema Migration im Wahlprogramm 2025?

Wahl 2025

Was sagt die FDP in ihrem Wahlprogramm zum Thema Digitalisierung?

Wahl 2025

Vergleiche die Positionen von AFD und Grünen zur Klimapolitik.

Wahl 2025

Worauf liegt der Fokus des Wahlprogramms der BSW?

Mit der Dokumentensuche

Nenne 5 Mitglieder der SPD und ihre Nebentätigkeiten.

Wie ist das Abstimmungsverhalten von Friedrich Merz beim Thema Energie?

Vergleiche das Abstimmungsverhalten von FDP und Grünen beim Thema Wirtschaft seit 2021 im Detail.

Erklären Sie mir das Gesetz mit dem Namen Fachkräfteeinwanderung.

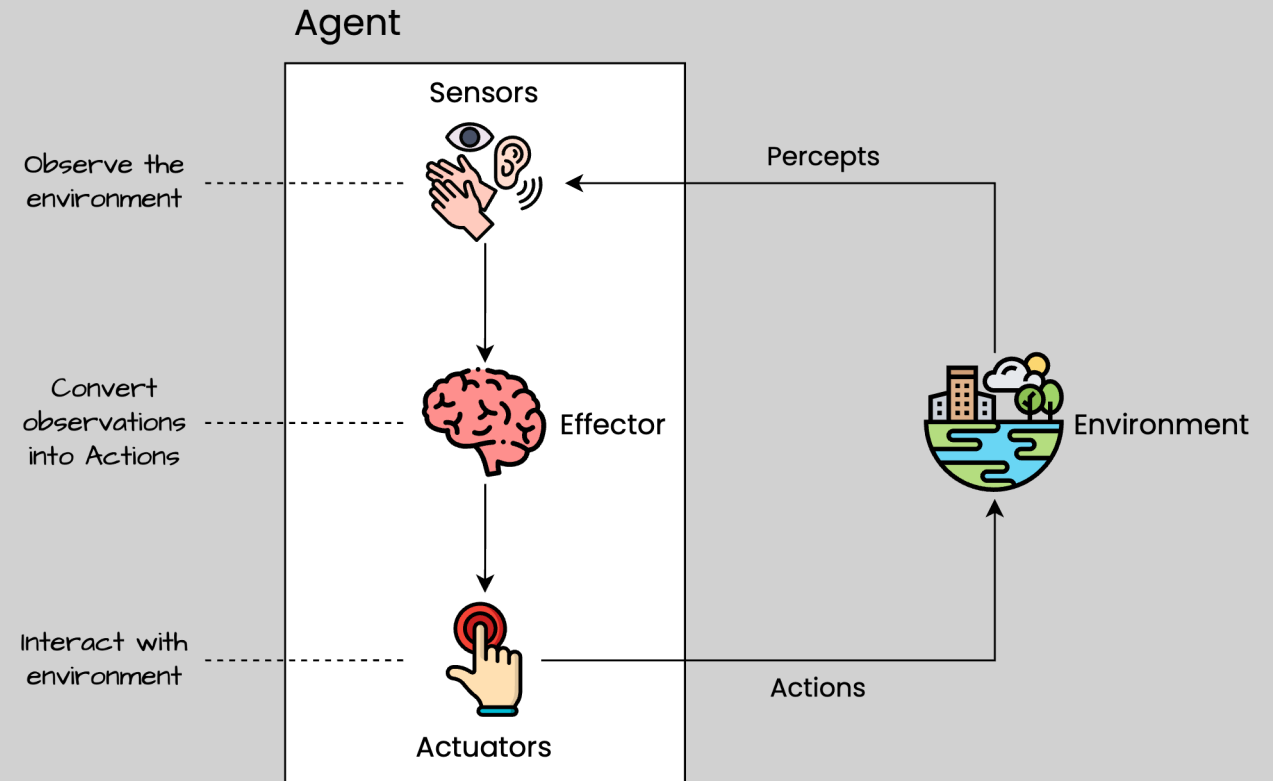
Stellen Sie Ihre Frage oder drücken Sie "/" für die spezialisierten Agenten ...

Senden

What is an Agent?

“An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.”

Russell & Norvig





Agent

Observe the environment

Percepts

What is an Agent?

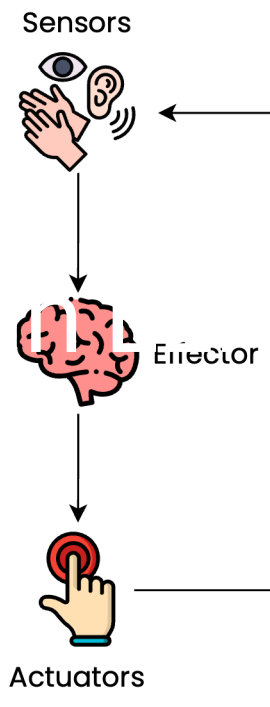
Observations into Actions

Effector

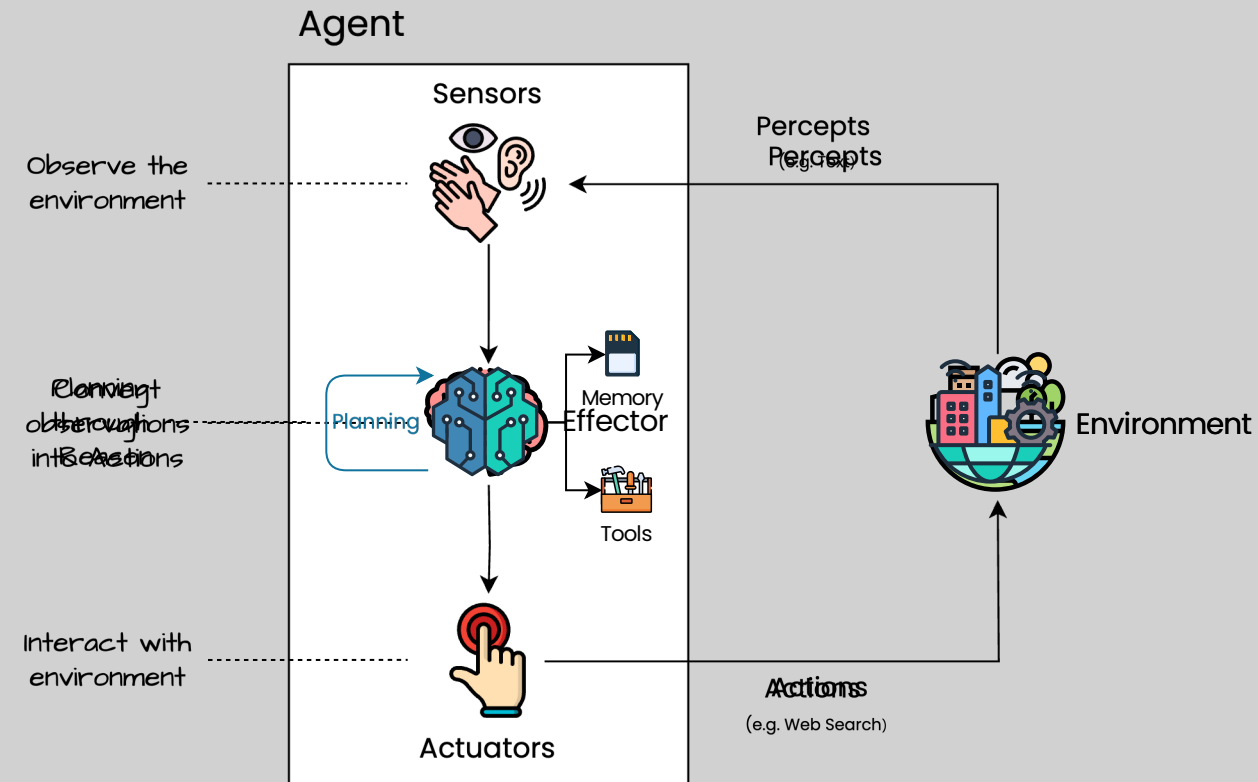
Environment

Interact with environment

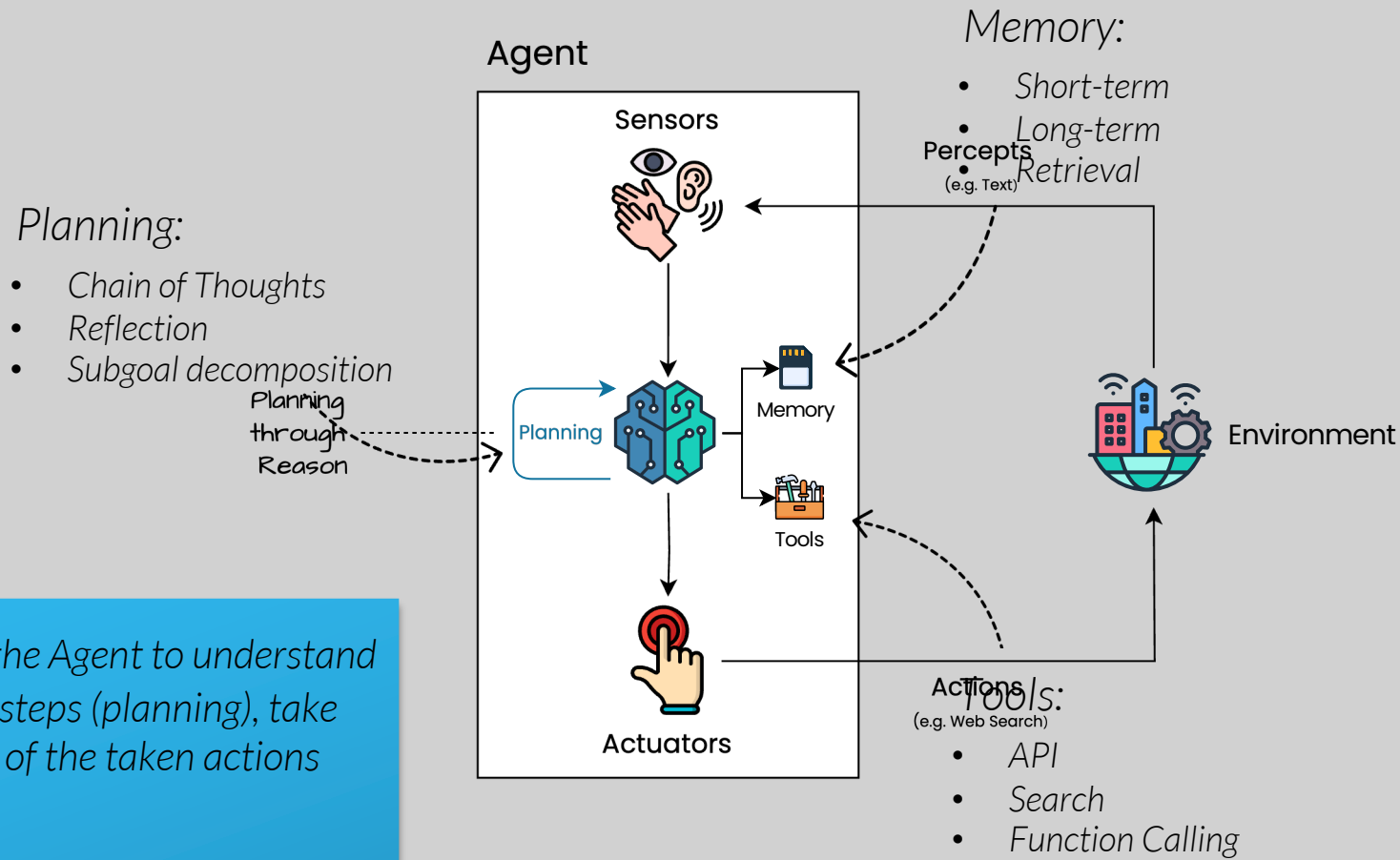
Action



What is an LLM Agent?



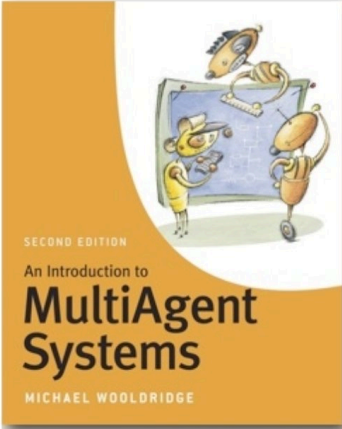
What is an LLM Agent?



“... Planning behavior allows the Agent to understand the situation (LLM), plan next steps (planning), take actions (tools), and keep track of the taken actions (memory).”

Marteen Grootendorst

Agents are nothing new!



An Introduction to MultiAgent Systems - Second Edition
by Michael Wooldridge

Published May 2009
by John Wiley & Sons

ISBN-10: 0470519460
ISBN-13: 978-0470519462

Multiagent systems are a new paradigm for understanding and building distributed systems, where it is assumed that the computational components are autonomous: able to control their own behaviour in the furtherance of their own goals. The first edition of *An Introduction to Multiagent Systems* was the first contemporary textbook in the area, and became the standard undergraduate reference work for the field. This second edition has been extended with substantial new material on recent developments in the field, and has been revised and updated throughout. It provides a comprehensive, coherent, and readable introduction to the theory and practice of multiagent systems, while presenting a wealth of discussion topics and pointers into more advanced issues for those wanting to dig deeper.

Key new features include:

- dedicated new chapters on ontologies, voting, auctions, bargaining, coalition formation, and argumentation, reflecting recent research directions and new results;
- "mind maps" to illustrate key concepts and ideas – an essential study and revision aid;
- 590 literature references, revised, updated, and extended to reflect the state of the art in agent research and development;

[Table of Contents](#)
[Resources for Teachers](#)
[Lecture Slides](#)
[Videos](#)
[Facebook Page](#)
[Endorsements](#)
[Translations](#)
[Legacy Site for First Edition](#)

- We think of an agent as being in a close-coupled, continual interaction with its environment:

sense – decide – act – sense – decide ...

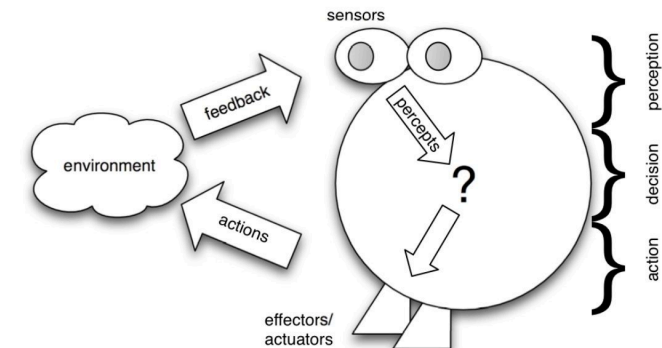
<http://www.csc.liv.ac.uk/~mjw/pubs/imas/>

1

Chapter 2

An Introduction to Multiagent Systems 2e

Agent and Environment



<http://www.csc.liv.ac.uk/~mjw/pubs/imas/>

2

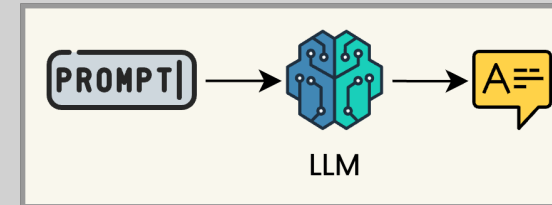
Degree of Autonomy

"... depending on who you ask, a system is more "agentic" the more the LLM decides how the system can behave."

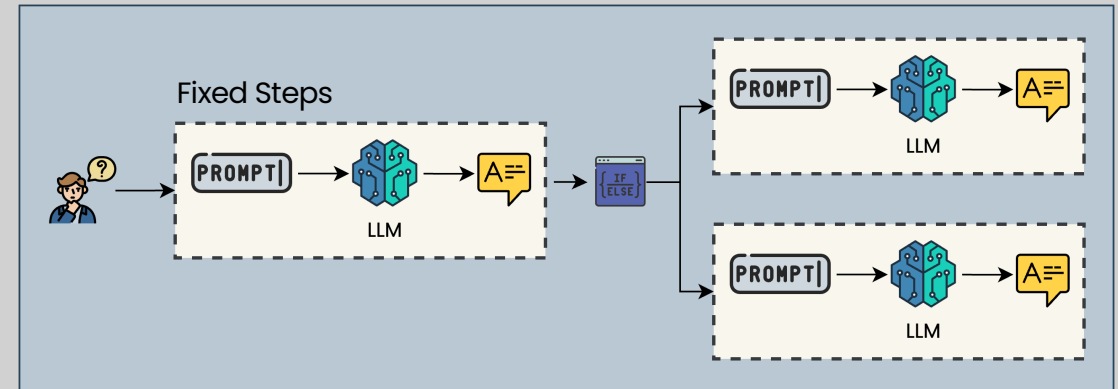
Marteen Grootendorst

They are commonly known as LLM Agents because of their ability to operate autonomously.

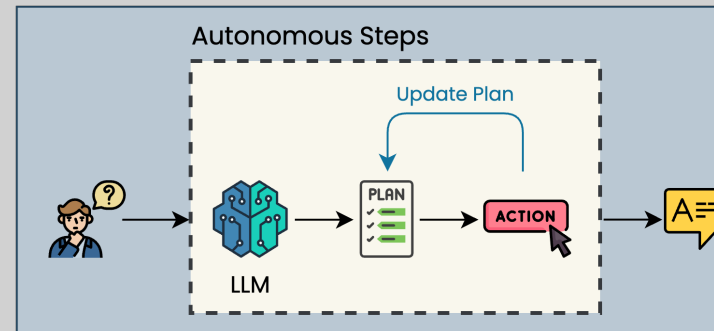
Simple Call



Chains



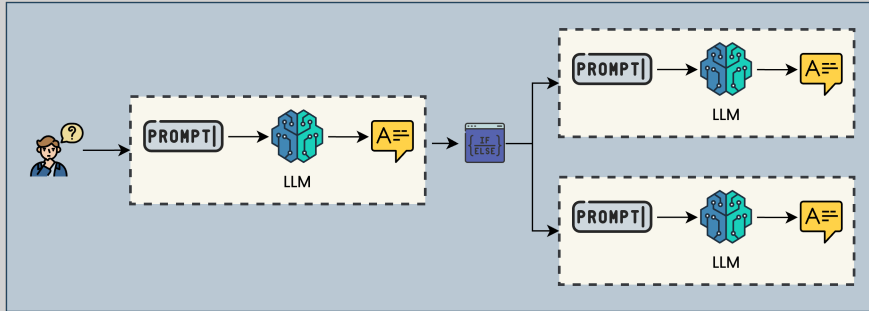
Agent



degree of
autonomy



Structure vs. Autonomy*



Well-understood Process



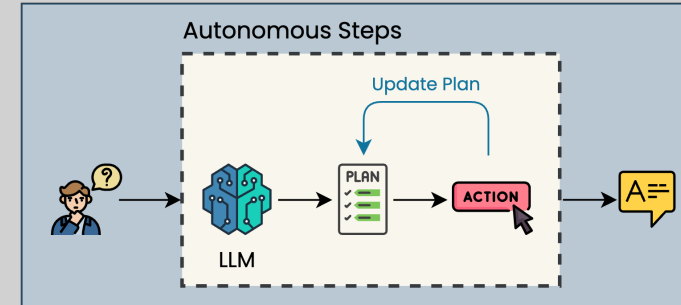
Critical Business Logic



Error-prone Operations



Structured Output



Unstructured Output



Flexible Ruleset



Handling Novel Situation

Agentic Frameworks



Easy to learn



Model-Agnostic

LangGraph



Rich Set of Built-in Tools



Built-in Memory Management

memory

Agno



PydanticAI



Comprehensive Documentation and Examples



Easy to Implement Multi-Agent Patterns

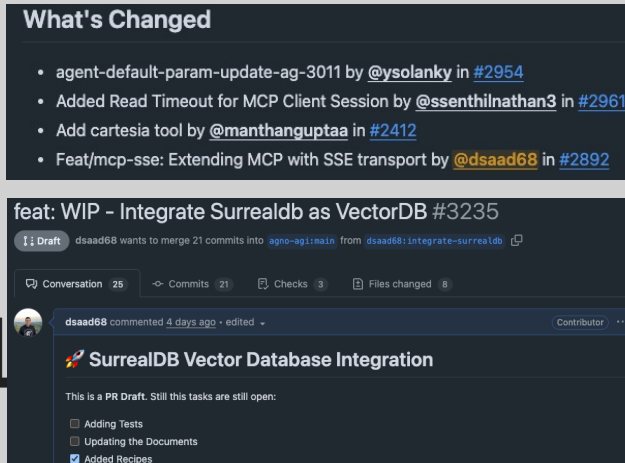


smolagents

LLamaIndex



Also, I contribute to this project



Why Agno?



Let's create our first agent with Agno

Which LLM model
must be used.

Structure the output
as Markdown.

gemini.py

— □ ×
_Agno

```
from agno.agent import Agent
from agno.models.google import Gemini

# Using Google AI Studio
agent = Agent(
    model=Gemini(id="gemini-2.0-flash"),
    markdown=True,
)

# Print the response in the terminal
agent.print_response("Share a 2 sentence horror story.")
```

Tools



AI GENERATED

What is a Tool?



Actions:

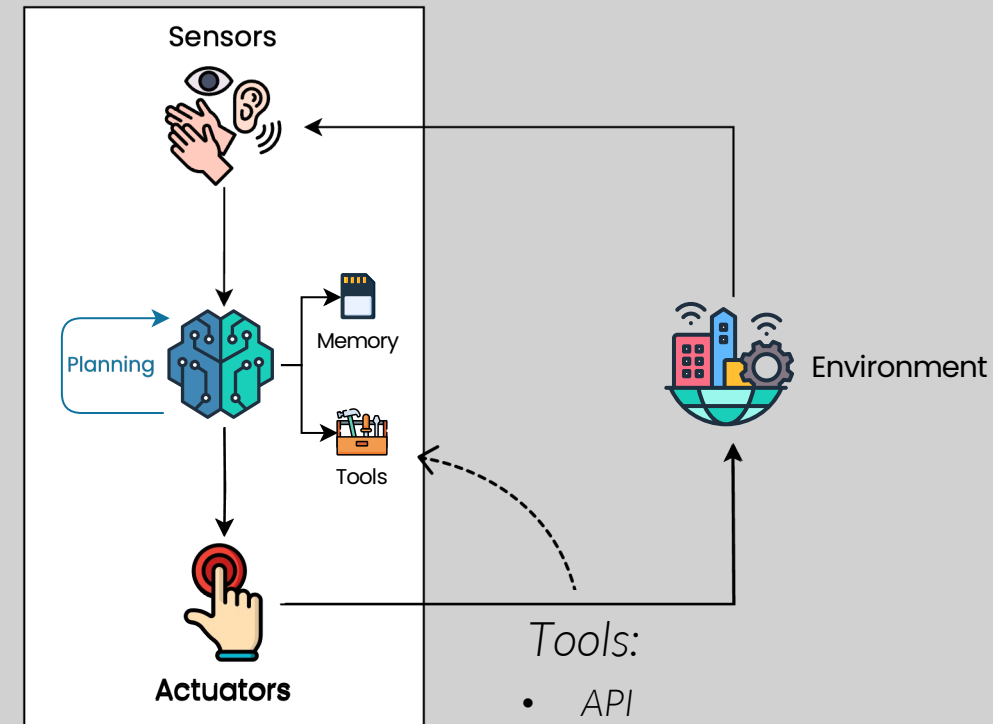
- Create Image
- Schedule a meeting
- Rewrite Content



Retrieve Data:

- Search
- Query Database
- Access Information

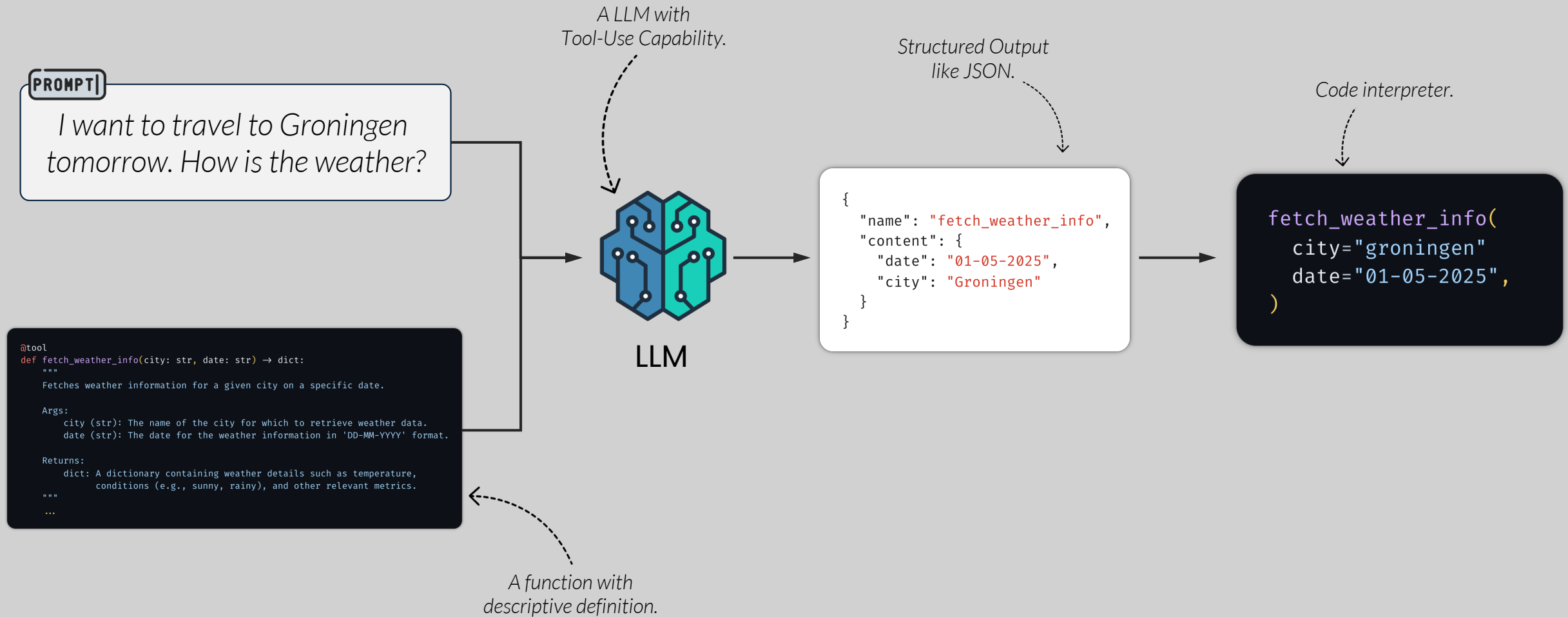
Agent



Tools:

- API
- Search
- Function Calling

How do Tools work?



How to use your Tools in Agno?

Defining the tool.

```
weather_tools.py

from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools import tool
```

```
@tool(show_result=True, stop_after_tool_call=True)
def fetch_weather_info(city: str, date: str) → dict:
    ...
```

Providing the tool to LLM.

```
agent = Agent(
    model=OpenAIChat(model="gpt-4o-mini"),
    tools=[get_weather],
    markdown=True,
)
```

```
agent.print_response("What is the weather in Groningen on 01-05-2025?",
                    stream=True)
```

How to use predefined Tools in Agno?

Providing the tool to LLM.

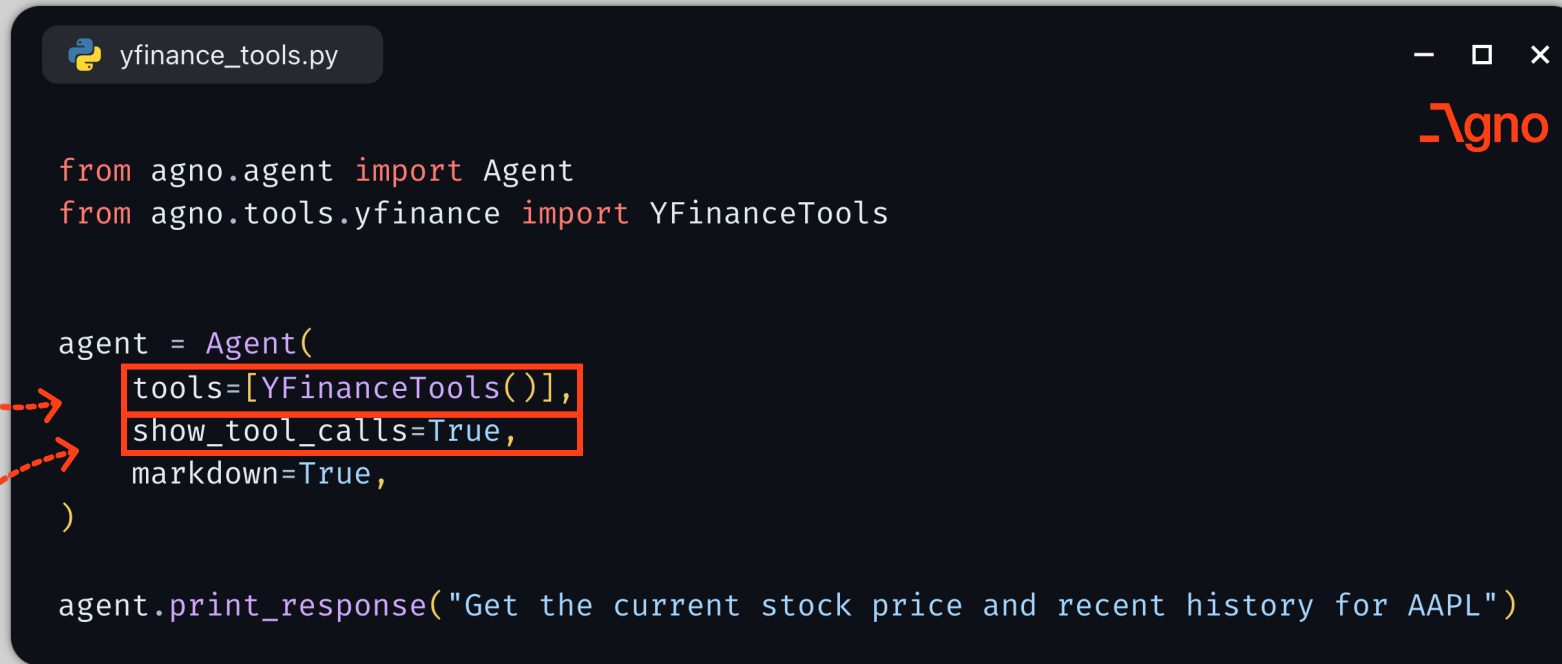
Show tools input & output.

```
yfinance_tools.py
```

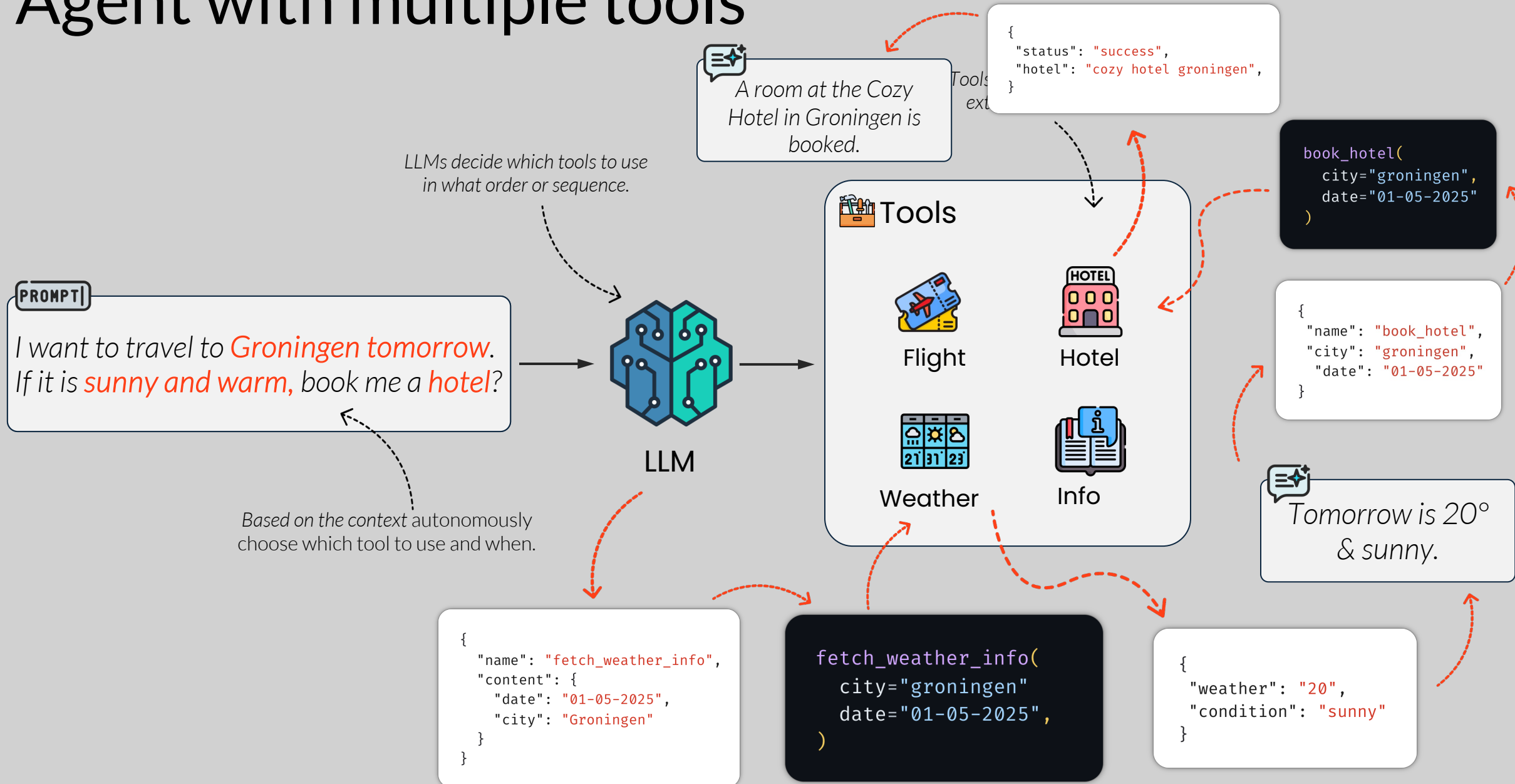
```
from agno.agent import Agent
from agno.tools.yfinance import YFinanceTools

agent = Agent(
    tools=[YFinanceTools()],
    show_tool_calls=True,
    markdown=True,
)

agent.print_response("Get the current stock price and recent history for AAPL")
```



Agent with multiple tools



Agno Agent with multiple tools

Providing the tools to LLM.

multiple_tools.py

— □ ×
_Agno

```
from agno.agent import Agent
from agno.tools.exa import ExaTools
from agno.tools.yfinance import YFinanceTools
```

```
agent = Agent(
    tools=[
        ExaTools(),
        YFinanceTools(),
    ],
    show_tool_calls=True,
    markdown=True,
)
```

```
agent.print_response("Get the news and current stock price and recent history for AAPL")
```

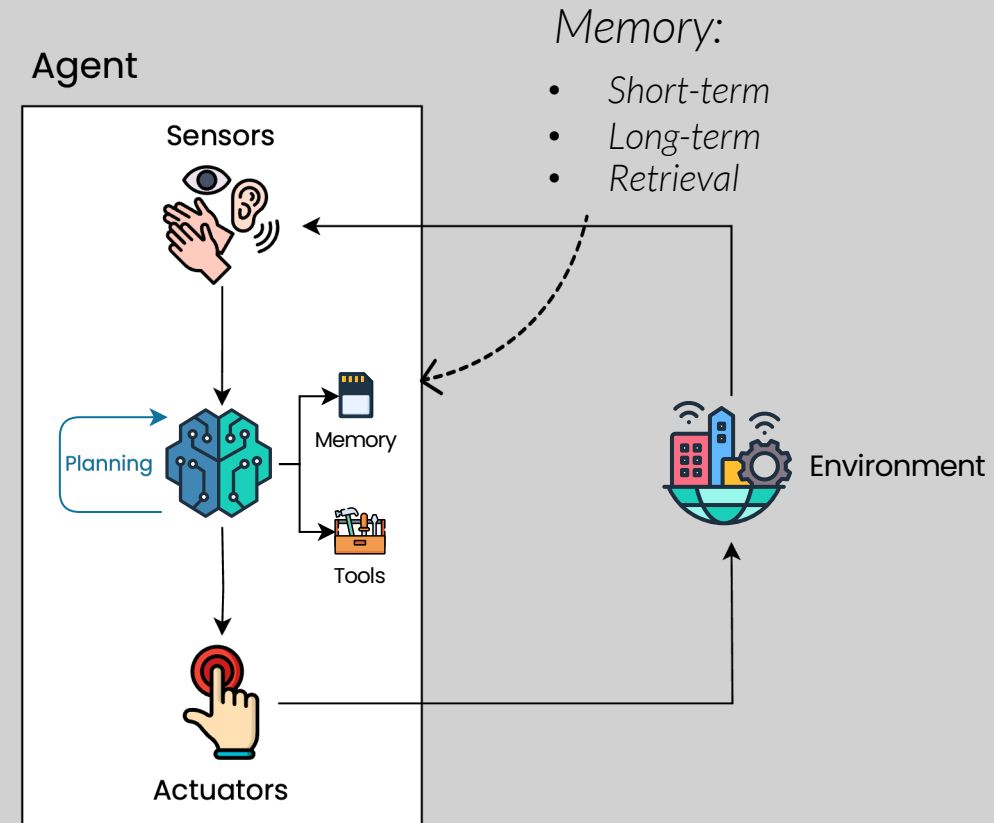
Memory

AI GENERATED

Memory



LLMs are forgetful.



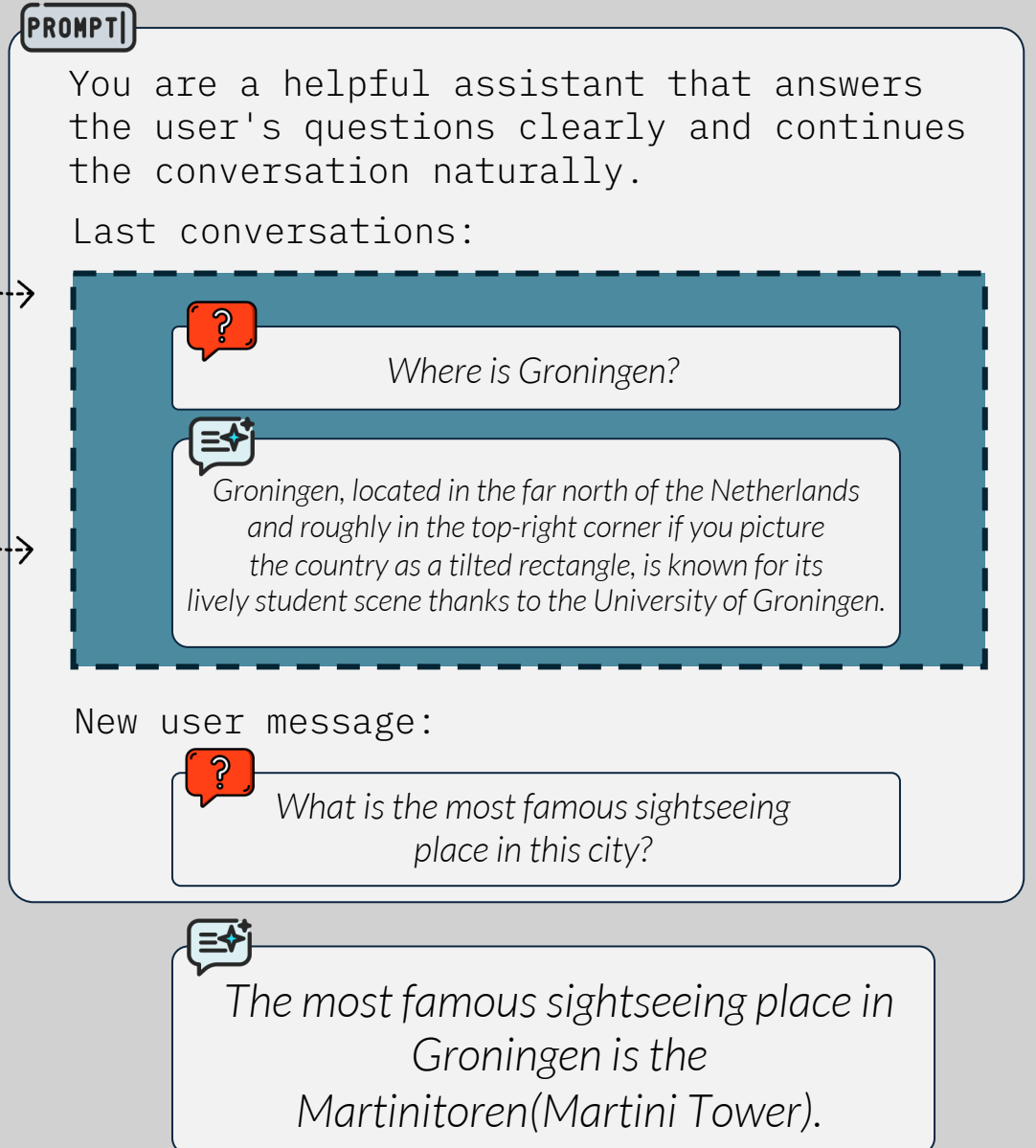
Short-term Memory



Utilizing the LLM's Context window for giving the Agent a memory.

If the conversation gets long, another technique is to add a summary of the conversation into the prompt.

Independent conversation.

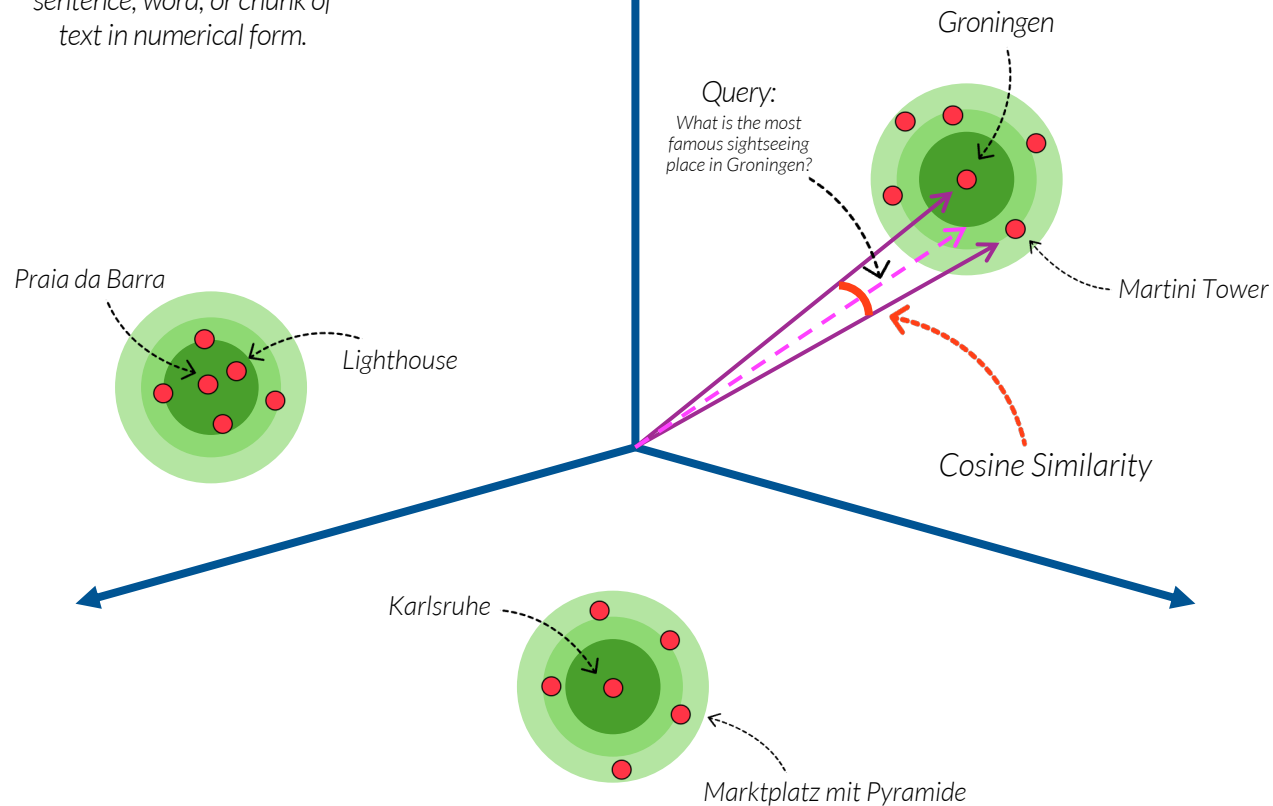


A Primer on the RAG

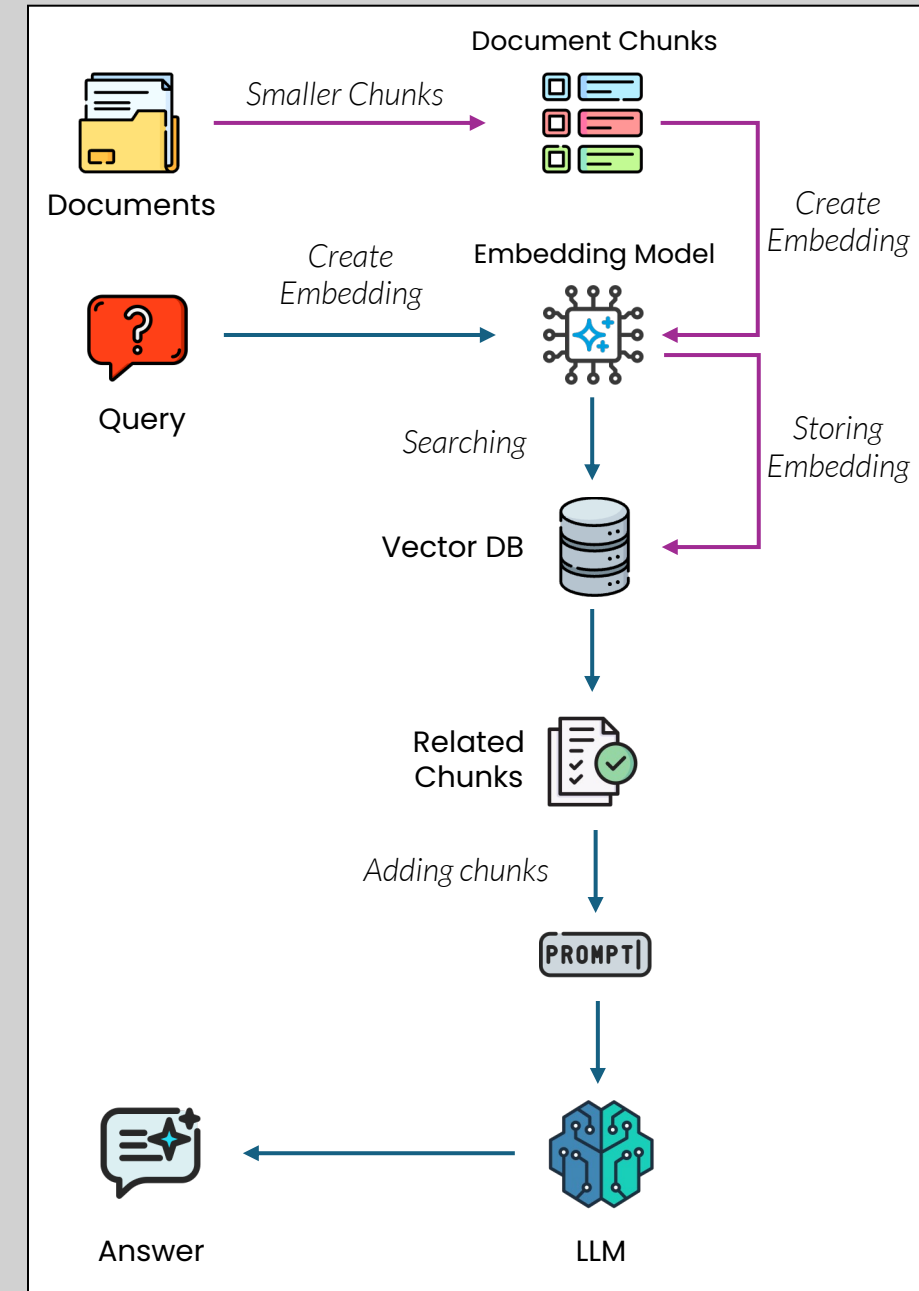
Retrieval-Augmented
Generation

Vector Similarity

Embedding:
A vector that captures the
semantic meaning of a
sentence, word, or chunk of
text in numerical form.



Retrieval-Augmented Generation



Sample RAG with Agno

Create a Vector DB.

```
# Create a LanceDB Vector DB
lance_vector_db = LanceDb(
    table_name="recipes",
    uri="/tmp/lancedb",
)
```

Create A Knowledge Base

```
pdf_knowledge_base = PDFKnowledgeBase(
    path="data/pdfs",
    vector_db=lance_vector_db,
    reader=PDFReader(chunk=True),
)
```

Giving access to Knowledge
base to the agent

```
agent = Agent(
    knowledge=knowledge_base,
    search_knowledge=True,
)
```

Loading the knowledge base.

```
# Loading the Knowledge
agent.knowledge.load()
```

```
agent.print_response("What is the recipe for Carbonara?")
```

Long-term Memory

Past Conversations



Find me a place to stay in Lisbon.



There are great hotels, hostels
& AirBnB options in Lisbon.
Which one do you prefer?



I always prefer to stay at a *hotel*.



I want to travel to Groningen tomorrow.
If it is sunny and warm,
book me an accommodation?

Embedding
& Storing

Found
Memory



Vector DB

Searching for a
memory related to the
new message.

PROMPT

You are a helpful assistant that answers
the user's questions clearly and continues
the conversation naturally.

Retrieved Memory based on the new Message:



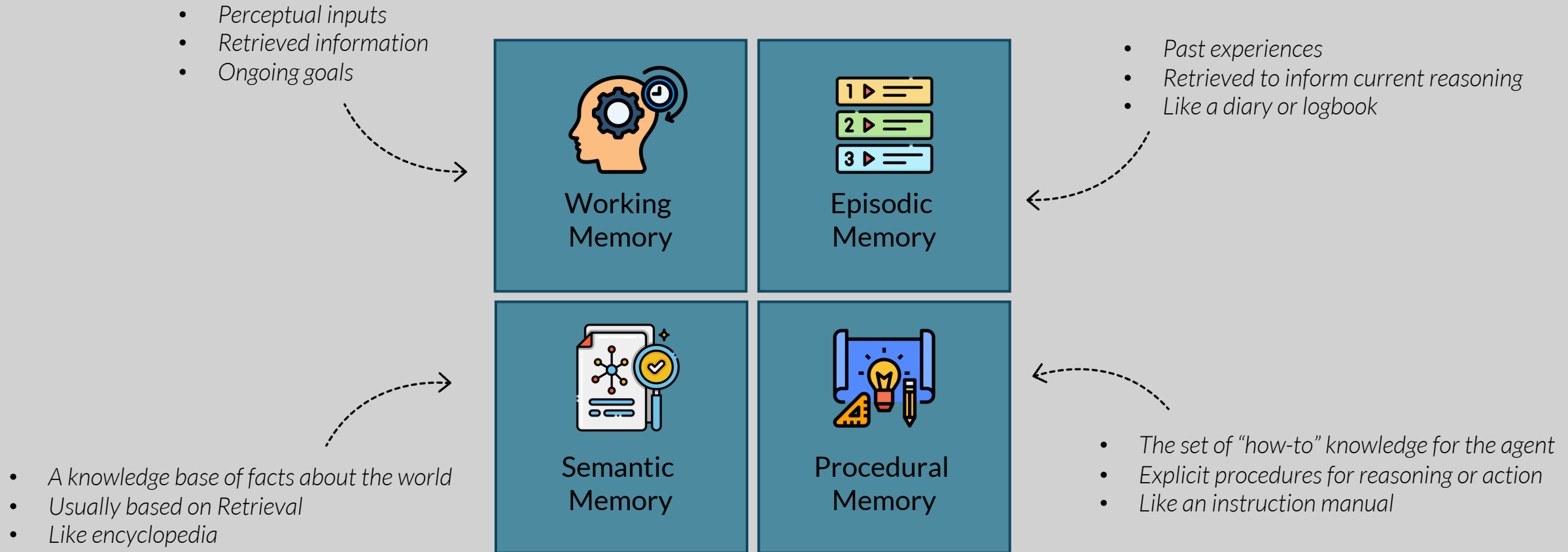
User prefers to stay at a hotel.

New user message:



I want to travel to Groningen tomorrow.
If it is sunny and warm,
book me an accommodation

Alternative Memory Classifications*



A black Mercedes-Benz G-Class SUV is positioned in the center of an ice hockey rink. The vehicle is a two-door model with a prominent Mercedes-Benz star emblem on the front grille. In the background, several ice hockey players are visible, some in white jerseys and others in red, engaged in a game. The rink's glass and metal barriers are visible in the distance.

Planning & Reasoning

AI GENERATED

Quick Primer on Prompt Engineering Techniques



Zero Shot

PROMPT

Is this movie review positive or negative?



Few Shot

PROMPT

Is this movie review positive or negative?

Examples:

Positive:

- *That movie was incredible!*
- *Such an amazing performance!*

Negative:

- *That film was a total mess.*
- *A complete waste of time.*

Planning

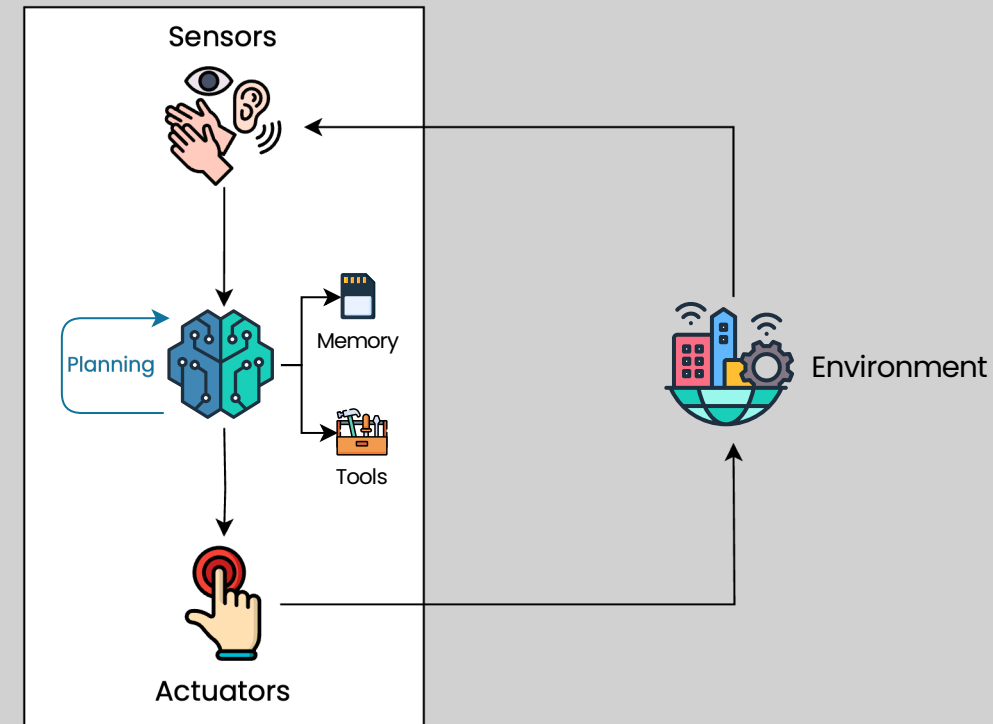


LLMs plan by breaking tasks into actionable steps, using task decomposition to split them into smaller, manageable tool uses or actions.

Planning:

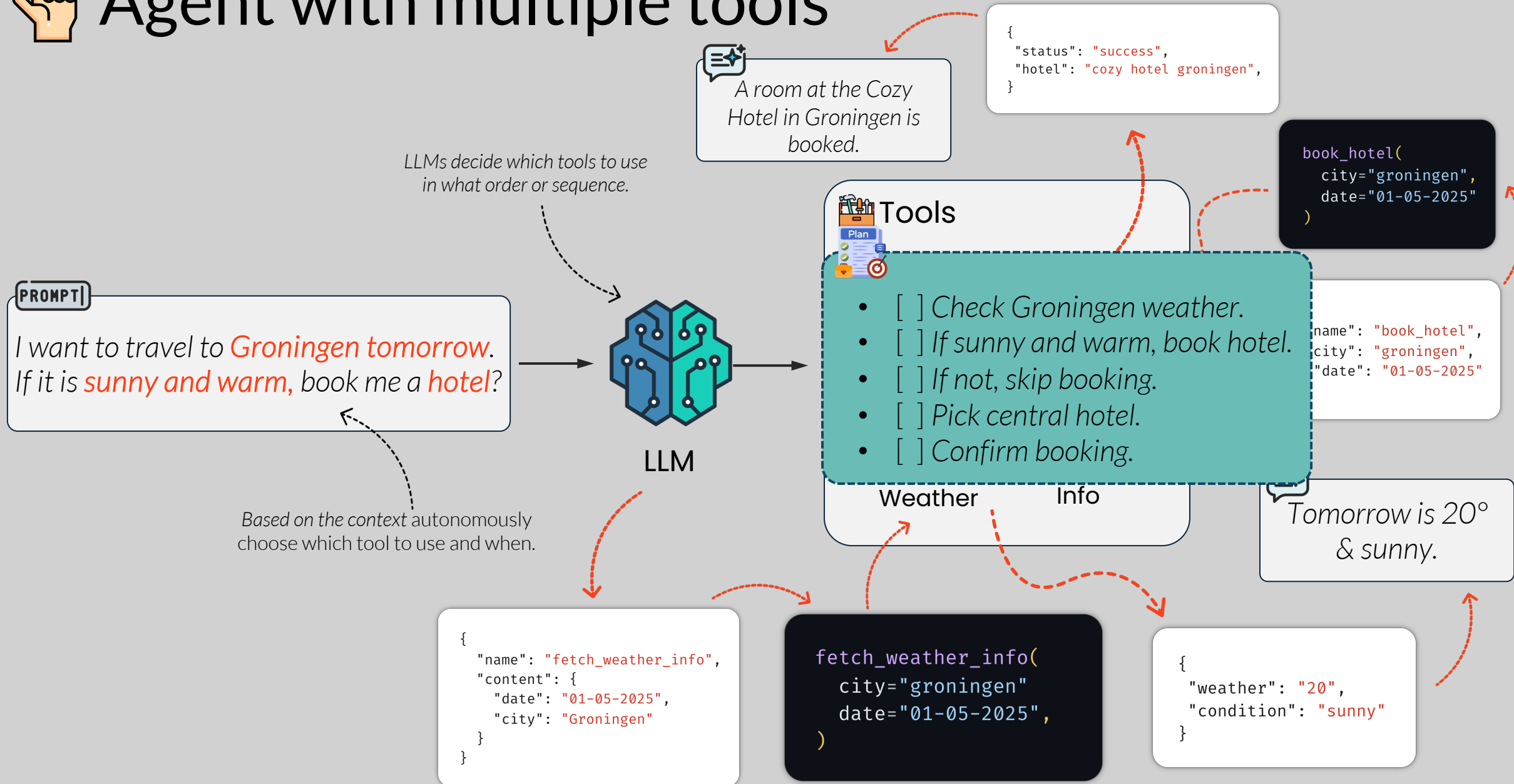
- Subgoal decomposition
- Chain of Thoughts
- Reasoning
- Reflection

Agent





Agent with multiple tools



Reasoning



Regular LLM

PROMPT

I had 2 apples. I got another 5 from my friend. I gave 3 to my sister. How many apples do I have now?



You have 4 🍏!



Reasoning LLM

PROMPT

I had 2 apples. I got another 5 from my friend. I gave 3 to my sister. How many apples do I have now?



2 ways this reasoning behavior:

- Fine-Tuning
- Specific Prompts



You have 2 apple.



You got 5. Now you have 7.



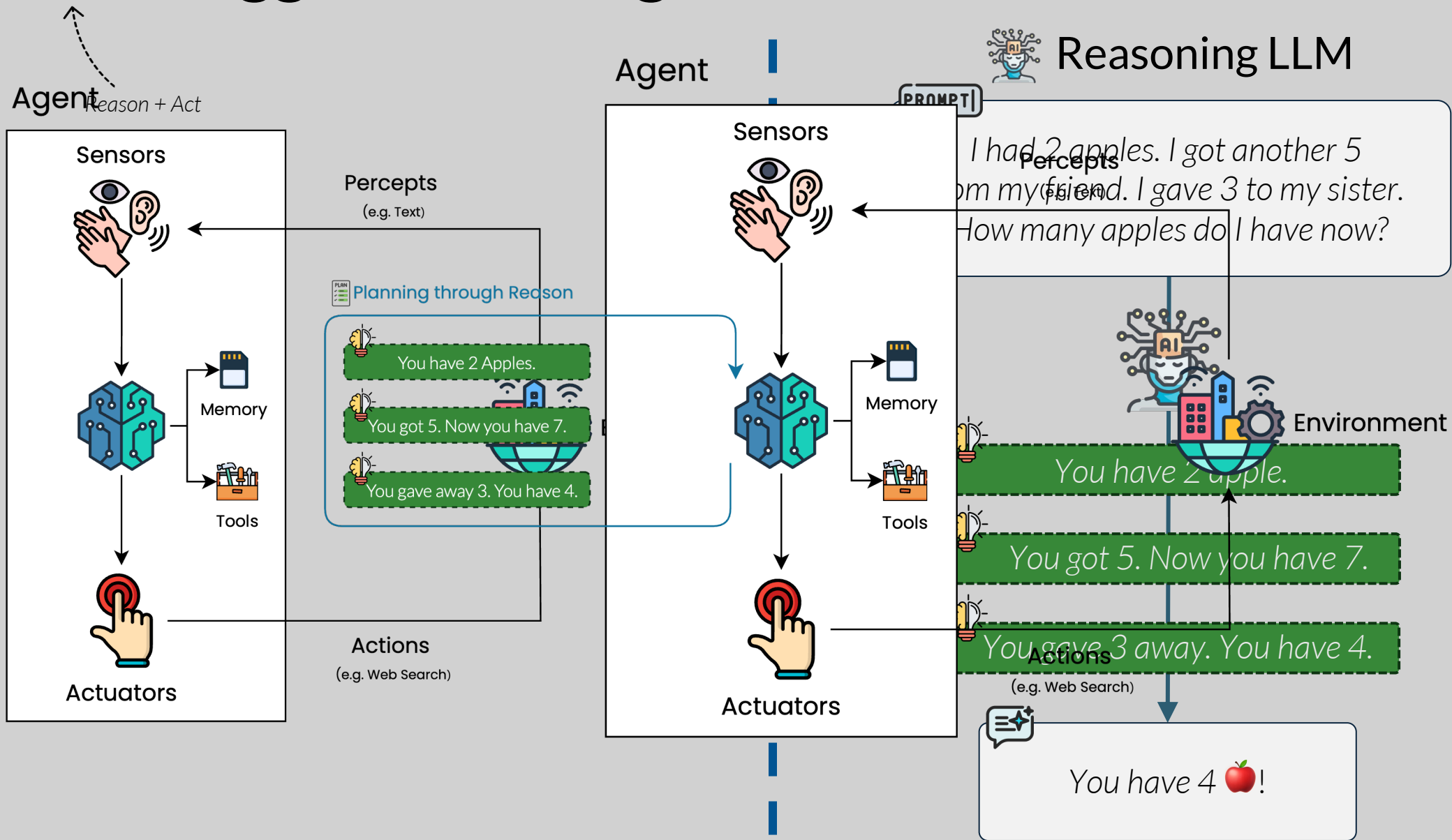
You gave 3 away. You have 4.



You have 4 🍏!

This is not much of thinking, it is much more of a problem or task decomposition.

Reasoning and Acting



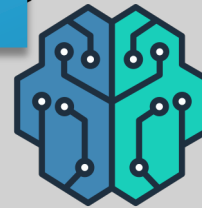


Agent with multiple tools

"It continues this behavior until an action specifies to return the result. By iterating over thoughts and observations, the LLM can plan out actions, observe its output, and adjust accordingly."

Marteen Grootendorst

I want to travel to **Groningen tomorrow**.
If it is **sunny and warm**, book me a **hotel**?



LLM



Tools



Flight



Weather



Hotel



Info



A room at the Cozy Hotel in Groningen is booked.

```
{
  "status": "success",
  "hotel": "cozy hotel groningen",
}
```

```
book_hotel(
  city="groningen",
  date="01-05-2025"
)
```

```
{
  "name": "book_hotel",
  "city": "groningen",
  "date": "01-05-2025"
}
```



Tomorrow is 20° & sunny.

Thought

LLM Output



I need to see how is the weather in Groningen.

```
{
  "name": "fetch_weather_info",
  "content": {
    "date": "01-05-2025",
    "city": "Groningen"
  }
}
```

```
fetch_weather_info(
  city="groningen"
  date="01-05-2025",
)
```

```
{
  "weather": "20",
  "condition": "sunny"
}
```

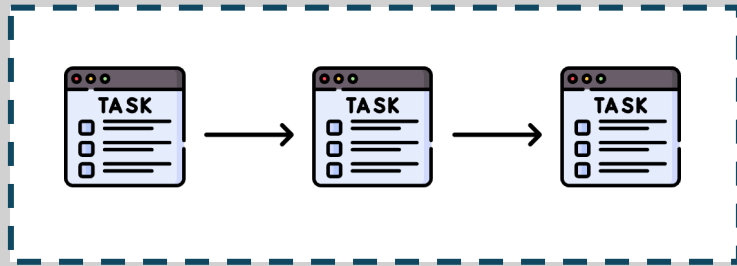
Based on the context autonomously choose which tool to use and when.

Action

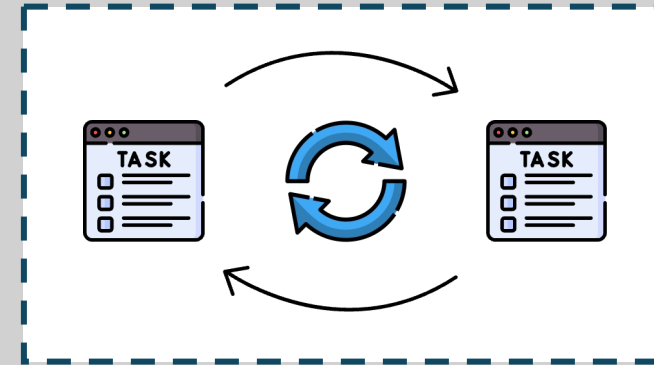
Observe

Complex Planning*

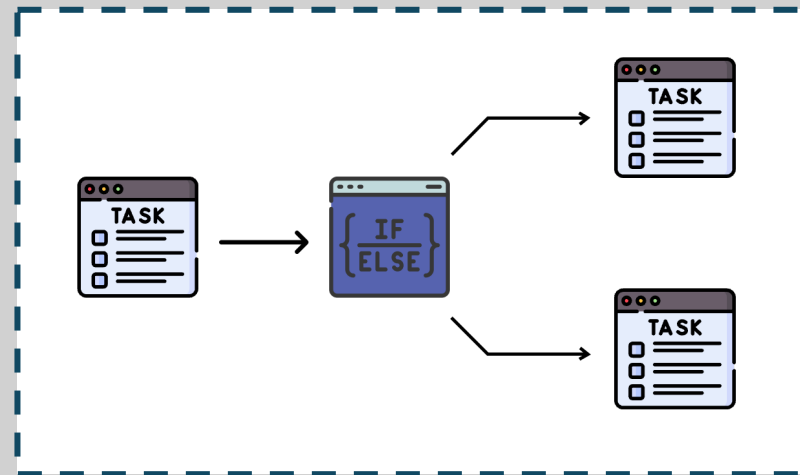
Sequential



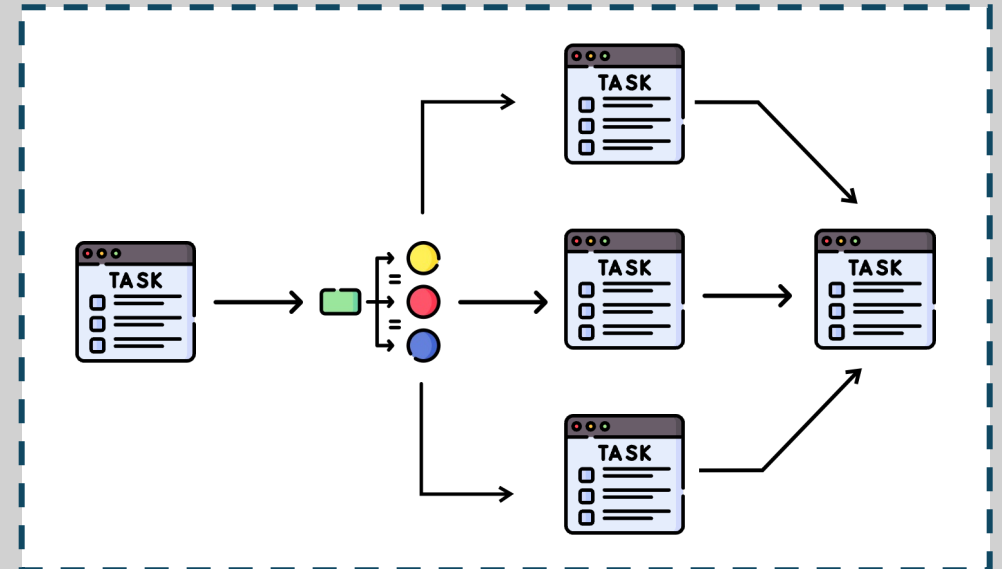
Loop



Conditional



Map Reduce



Reflecting



ReAct Agents are not immune to errors.



Approaches:

- SELF-REFINE
- REFLEXION

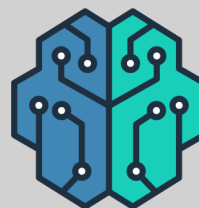
SELF-REFINE*



Write a Fibonacci function in python?



✓ Very fast and acceptable answer!



LLM



```
def fibonacci_slow(n):  
    if n ≤ 1:  
        return n  
    return fibonacci_slow(n-1) + fibonacci_slow(n-2)
```



✗ Very slow for large n because:

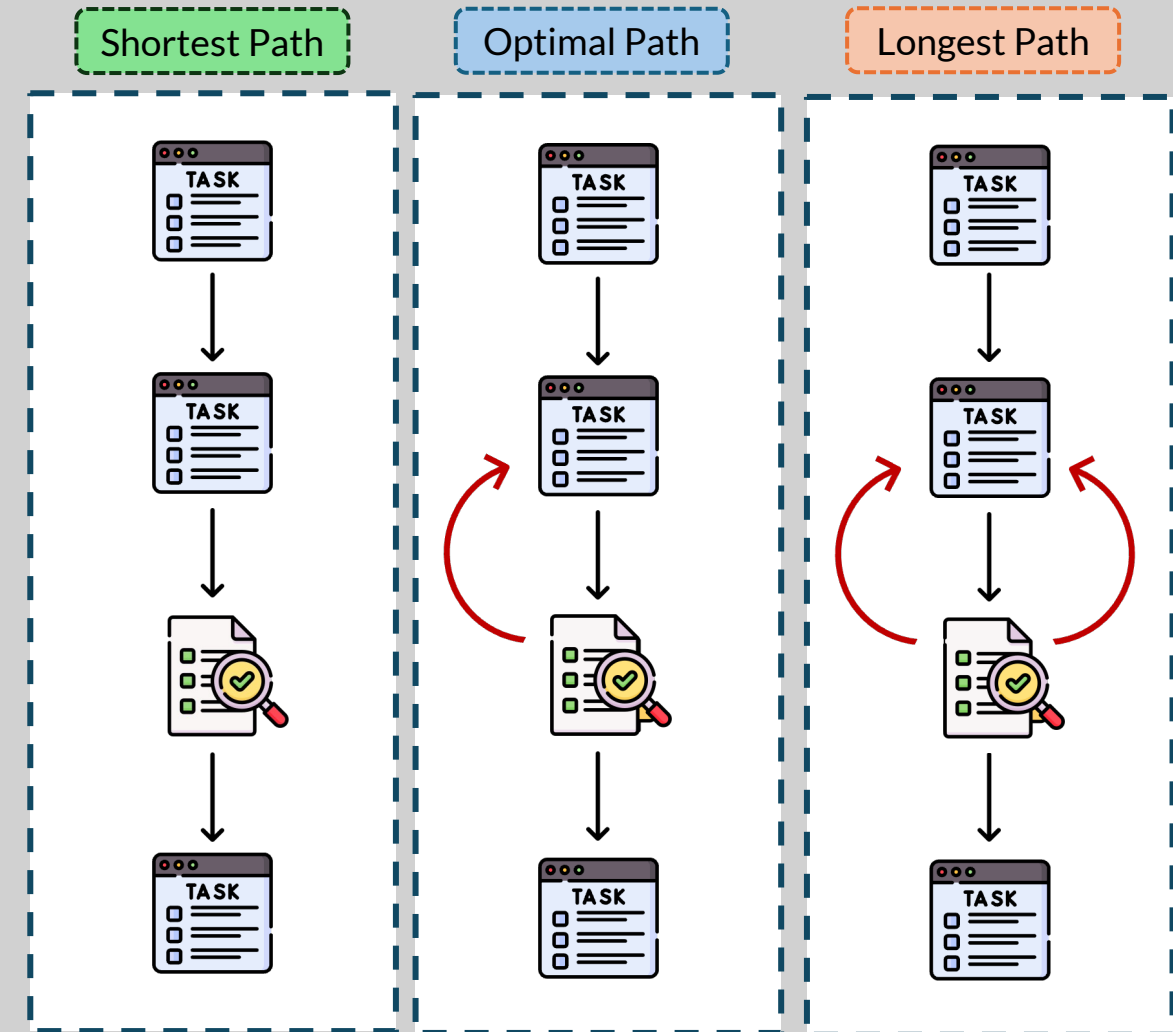
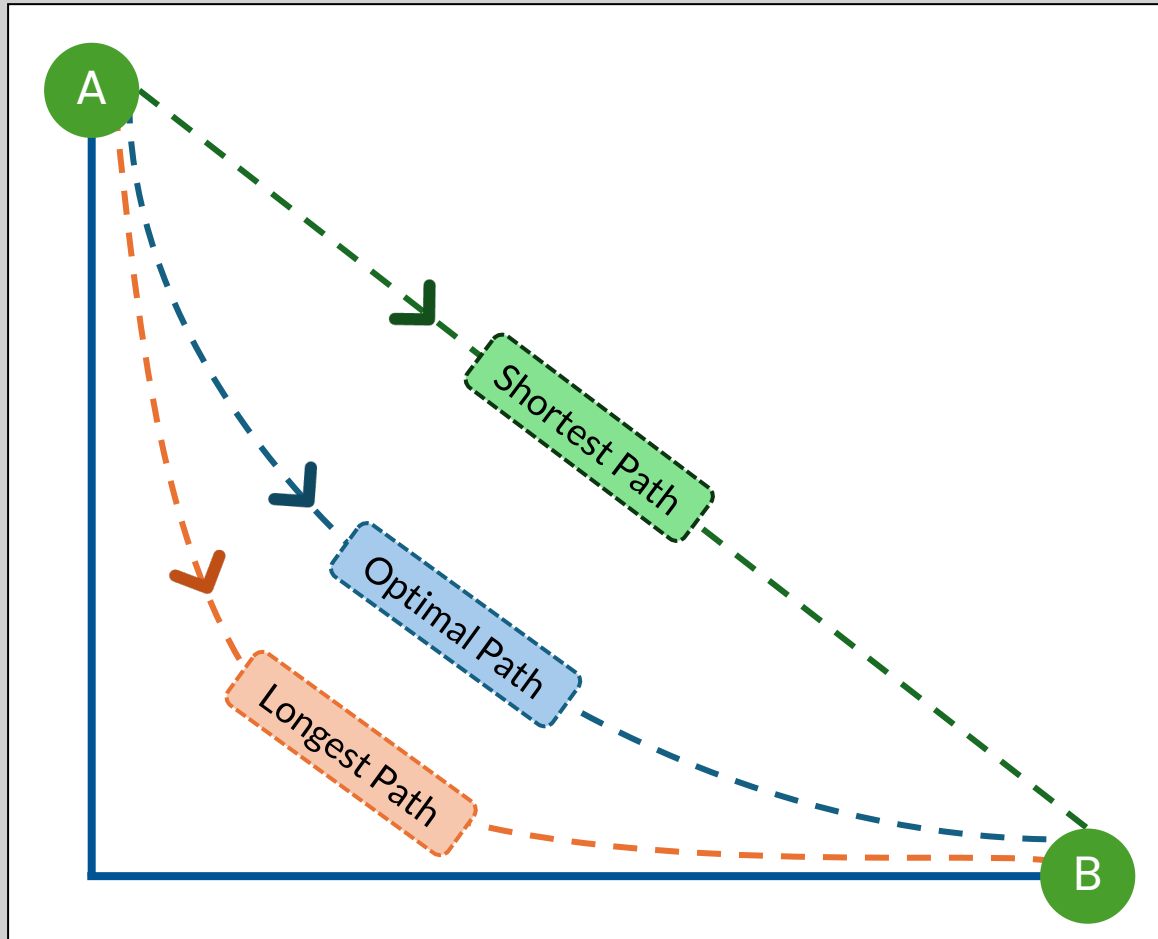
- It recomputes the same Fibonacci numbers repeatedly.
- Time Complexity: $O(2^n)$ (exponential time).



```
def fibonacci_fast(n, memo={0: 0, 1: 1}):  
    if n in memo:  
        return memo[n]  
    memo[n] = fibonacci_fast(n-1, memo) + fibonacci_fast(n-2, memo)  
    return memo[n]
```

Trajectory as an Agents' Evaluation Metric^{*}

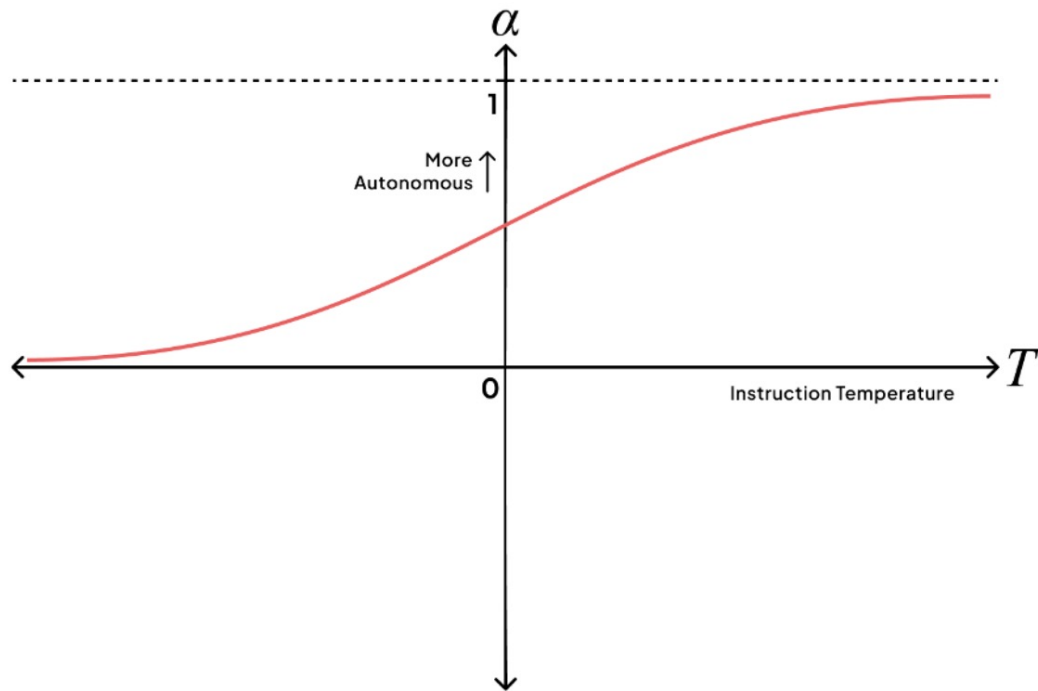
Trajectory from point A to Point B to Achieve Goal



^{*} <https://github.com/langchain-ai/agentevals>

Trajectory as an Agents' Evaluation Metric

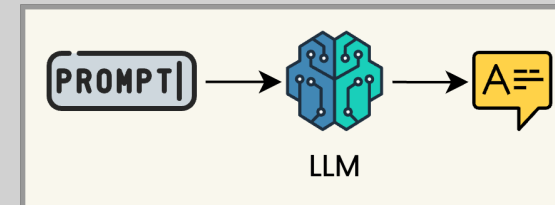
degree of
autonomy



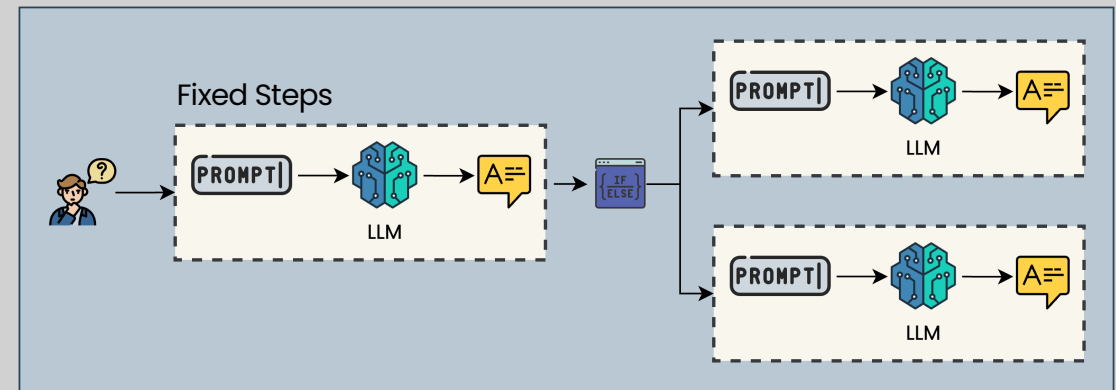
. The balance between the agent's autonomy and the influence of directives as a function of the instruction temperature. *

* Recursive Agent Trajectory Fine-Tuning: Utilizing Agent Instructions for Enhanced Autonomy and Efficiency in AI Agents

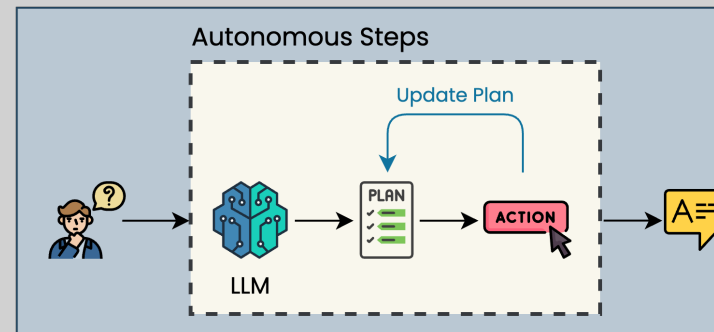
Simple Call



Chains



Agent



Reasoning Models



Reasoning models are trained with a large amount of data through a process of thought.

Using a reasoning Model.

- OpenAI
- Claude
- Gemini
- DeepSeek



This model solves the problem.

reasoning_models.py



```
from agno.agent import Agent
from agno.models.google import Gemini
from agno.tools.yfinance import YFinanceTools

agent = Agent(
    model=Gemini(id="gemini-2.0-flash-thinking-exp-1219"),
    tools=[YFinanceTools(stock_price=True,
                        analyst_recommendations=True,
                        company_info=True,
                        company_news=True)],
    instructions="Use tables to display data.",
    show_tool_calls=True,
    markdown=True,
)

agent.print_response("Write a report comparing NVDA to TSLA", stream=True)
```


Reasoning Tools



A new area of research is emerging, showing that giving models tools for structured thinking, such as a scratchpad, significantly enhances their reasoning abilities.



LLM



Tools



Flight



Hotel



Reasoning

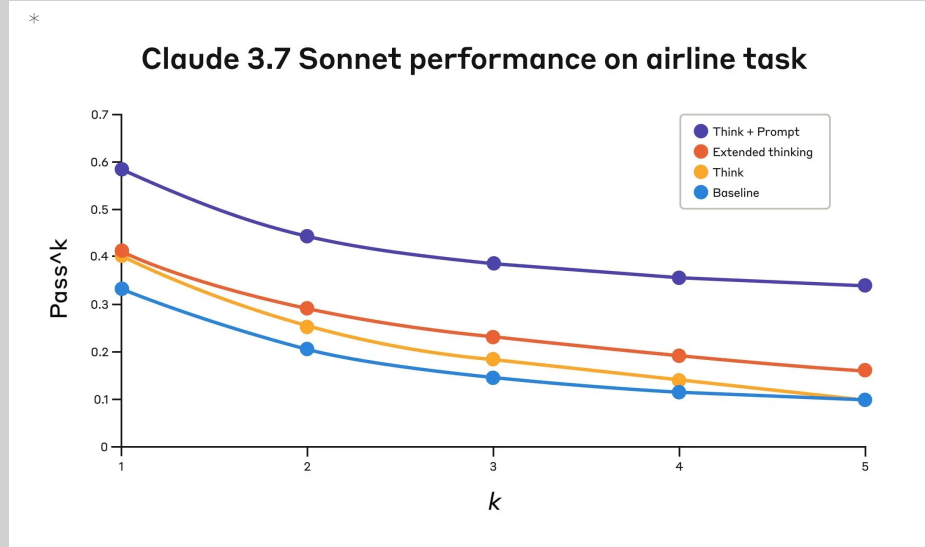


Weather



Info

Pass^k measures the probability that an AI agent succeeds on all k independent attempts.



Reasoning Tools

Providing the reasoning tool to LLM.

Show the Reasoning output.

```
reasoning_tool.py

from agno.agent import Agent
from agno.models.google import Gemini
from agno.tools.reasoning import ReasoningTools
from agno.tools.yfinance import YFinanceTools

reasoning_agent = Agent(
    model=Gemini(id="gemini-2.5-pro-preview-03-25"),
    tools=[
        ReasoningTools(
            think=True,
            analyze=True,
            add_instructions=True,
        ),
        YFinanceTools(
            stock_price=True,
            analyst_recommendations=True,
            company_info=True,
            company_news=True,
        ),
    ],
    instructions="Use tables where possible",
    stream_intermediate_steps=True,
    show_tool_calls=True,
    markdown=True,
    debug_mode=True,
)

reasoning_agent.print_response(
    "Write a report comparing NVDA to TSLA.",
    show_full_reasoning=True
)
```



Agno's Reasoning Agent



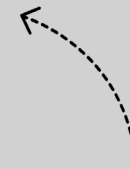
Reasoning Agents are a new type of multi-agent system developed by Agno that combines chain of thought reasoning with tool use.



I want to travel to **Groningen tomorrow**.
If it is **sunny and warm**, book me a **hotel**?



Reasoning
Agent



A separate "Reasoning Agent" to first solve the problem using a chain-of-thought.

Agno's Reasoning Agent

Enabling the
reasoning agent.

reasoning_agent.py

Agno

```
from agno.agent import Agent
from agno.models.google import Gemini
from agno.tools.yfinance import YFinanceTools

agent = Agent(
    model=Gemini(id="gemini-2.0-flash"),
    tools=[YFinanceTools(stock_price=True,
                        analyst_recommendations=True,
                        company_info=True,
                        company_news=True)],
    instructions="Use tables to display data.",
    reasoning=True,
    show_tool_calls=True,
    markdown=True,
)

agent.print_response("Write a report comparing NVDA to TSLA", stream=True)
```



Multi-Agent Systems

AI GENERATED

Multi-Agent Systems



Multi-agent systems consist of multiple agents working together to solve complex tasks.



Agents can be organized using a variety of design patterns, each with its own strengths and weaknesses.



Agents



Flight



Hotel



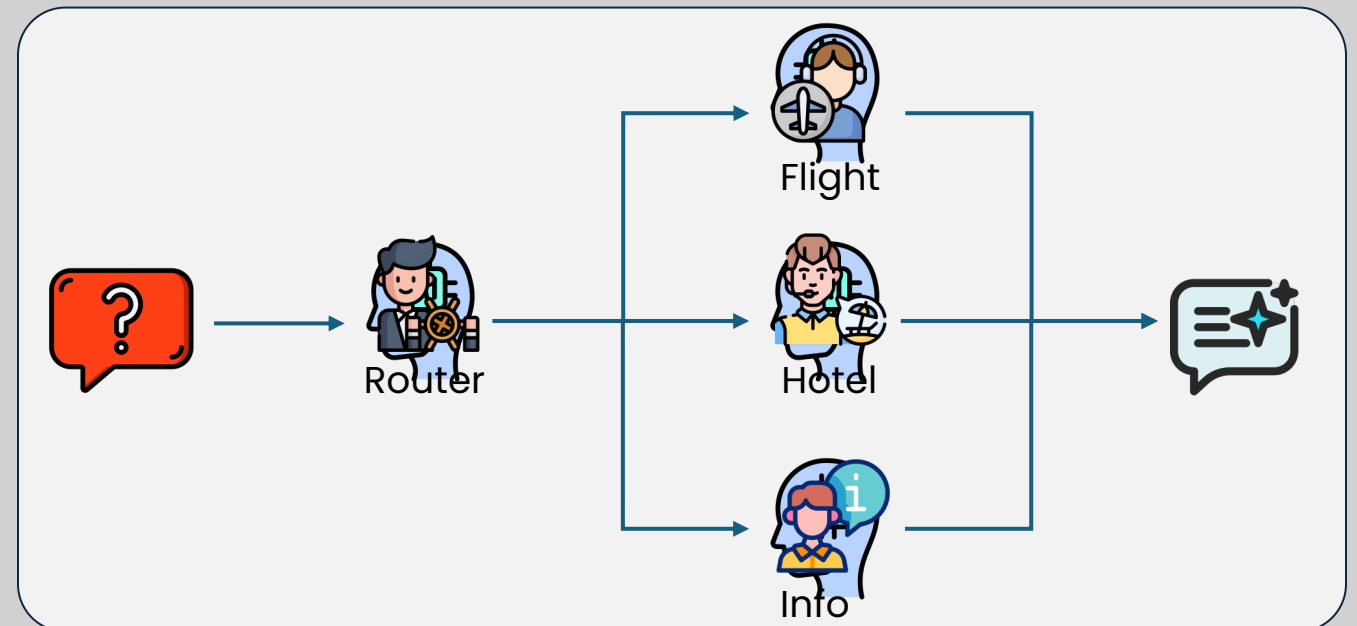
Weather



Info

Router

 A router agent determines which agents to invoke based on the task or input.



Router

Defining an Agent Team member.

Each agent can use a different model.

Defining the Router Agent.

Defining an Agent Team.

Showing output of members.

```
router.py
from agno.agent import Agent
from agno.models.anthropic import Claude
from agno.models.deepseek import DeepSeek
from agno.models.mistral.mistral import MistralChat
from agno.models.openai import OpenAIChat
from agno.team.team import Team

english_agent = Agent(
    name="English Agent",
    role="You can only answer in English",
    model=OpenAIChat(id="gpt-4.5-preview"),
    instructions=[
        "You must only respond in English",
    ],
)

japanese_agent = Agent(
    name="Japanese Agent",
    role="You can only answer in Japanese",
    model=DeepSeek(id="deepseek-chat"),
    instructions=[
        "You must only respond in Japanese",
    ],
)

german_agent = Agent(
    name="German Agent",
    role="You can only answer in German",
    model=Claude("claude-3-5-sonnet-20241022"),
    instructions=[
        "You must only respond in German",
    ],
)

multi_language_team = Team(
    name="Multi Language Team",
    mode="route",
    model=OpenAIChat("gpt-4.5-preview"),
    members=[
        english_agent,
        japanese_agent,
        german_agent,
    ],
    show_tool_calls=True,
    markdown=True,
    instructions=[
        "You are a language router that directs questions to the appropriate language agent.",
        "If the user asks in a language whose agent is not a team member, respond in English with:",
        "I can only answer in the following languages: English, Japanese and German.",
        "Please ask your question in one of these languages.",
        "Always check the language of the user's input before routing to an agent.",
        "For unsupported languages like Italian, respond in English with the above message.",
    ],
    show_members_responses=True,
)

multi_language_team.print_response(
    "お元気ですか?", stream=True # Japanese
)
```

Router Instructions.

Supervisor

*It has other names:
Hierarchical, Vertical or
Coordinator.*

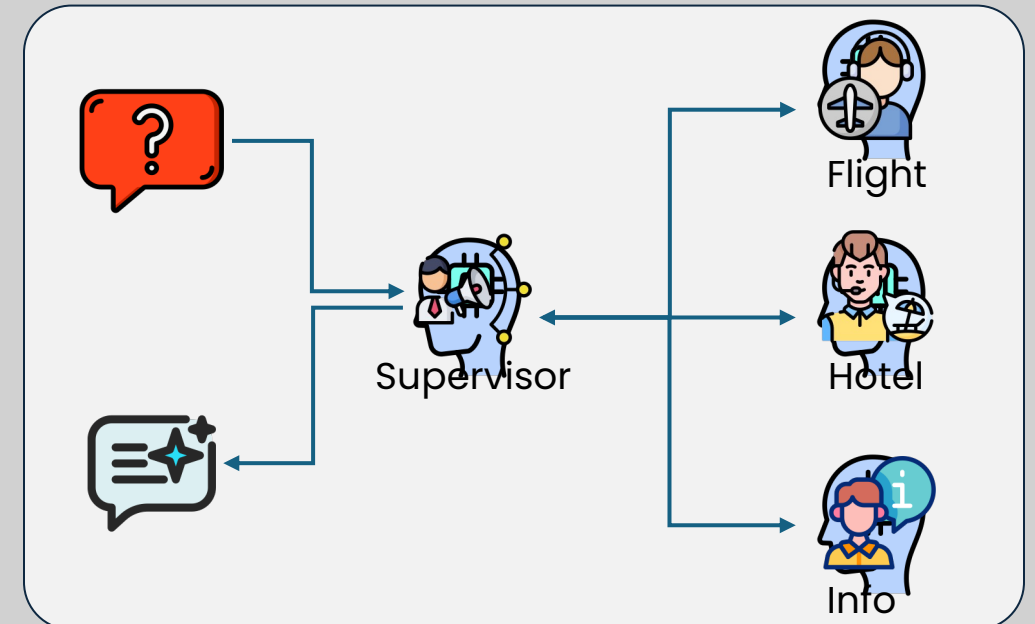


A supervisor agent manages and coordinates communication and state between other low-level agents.



Advantages

- Clear division of roles and responsibilities among agents at different levels.
- Streamlined communication
- Suitable for large systems with a structured decision flow.



Supervisor Pattern in Agno

Defining each team member.

Each member has clear goal and task.

Creating a team and expected output.

```
supervisor.py

from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.team import Team
from agno.tools.duckduckgo import DuckDuckGoTools

# Create individual specialized agents
researcher = Agent(
    name="Researcher",
    role="Expert at finding information",
    tools=[DuckDuckGoTools()],
    model=OpenAIChat("gpt-4o"),
)

writer = Agent(
    name="Writer",
    role="Expert at writing clear, engaging content",
    model=OpenAIChat("gpt-4o"),
)

# Create a team with these agents
content_team = Team(
    name="Content Team",
    mode="coordinate",
    members=[researcher,
              writer],
    instructions="You are a team of researchers and writers that work together to create high-quality content.",
    model=OpenAIChat("gpt-4o"),
    show_members_responses=True,
)

# Run the team with a task
content_team.print_response("Create a short article about quantum computing")
```

Network



Agents communicate directly with one another in a many-to-many fashion, forming a decentralized network.



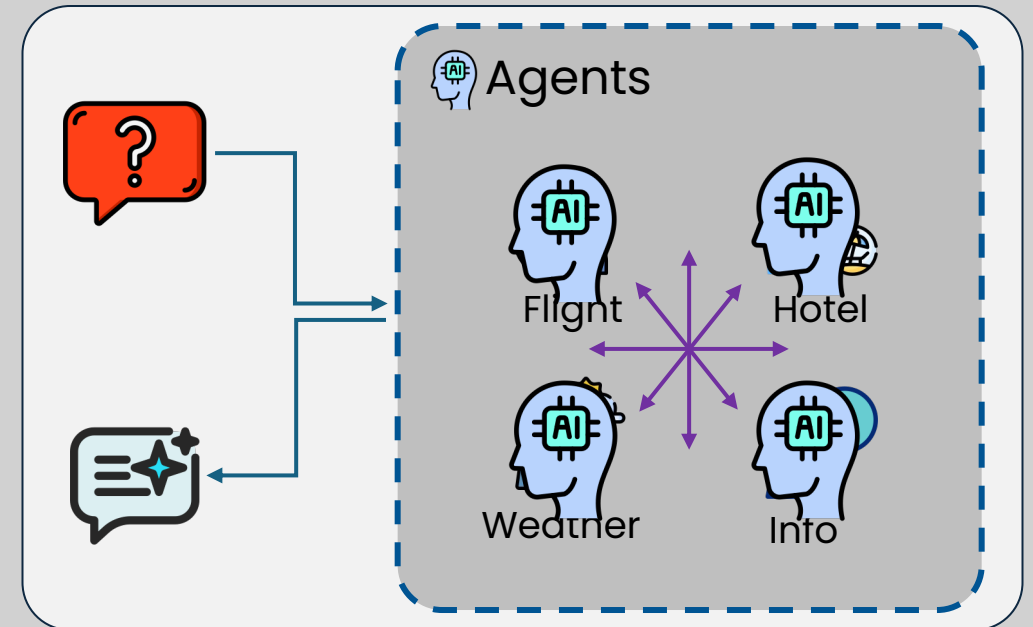
Advantages

- Distributed collaboration and group-driven decision-making



Disadvantages

- Managing communication among agents can become challenging
- More communication may cause inefficiencies
- and the possibility of agents duplicating efforts



Collaboration Pattern in Agno

Defining an agent with clear role, instruction and goal.

```
Collaborate.py

import asyncio
from textwrap import dedent

from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.team.team import Team
from agno.tools.arxiv import ArxivTools
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.tools.googlesearch import GoogleSearchTools
from agno.tools.hackernews import HackerNewsTools

reddit_researcher = Agent(
    name="Reddit Researcher",
    role="Research a topic on Reddit",
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGoTools()],
    add_name_to_instructions=True,
    instructions=dedent("""
    You are a Reddit researcher.
    You will be given a topic to research on Reddit.
    You will need to find the most relevant posts on Reddit.
    """),
)

academic_paper_researcher = Agent(
    name="Academic Paper Researcher",
    model=OpenAIChat("gpt-4o"),
    role="Research academic papers and scholarly content",
    tools=[GoogleSearchTools(), ArxivTools()],
    add_name_to_instructions=True,
    instructions=dedent("""
    You are a academic paper researcher.
    You will be given a topic to research in academic literature.
    You will need to find relevant scholarly articles, papers, and academic discussions.
    Focus on peer-reviewed content and citations from reputable sources.
    Provide brief summaries of key findings and methodologies.
    """),
)

agent_team = Team(
    name="Discussion Team",
    mode="collaborate",
    model=OpenAIChat("gpt-4o"),
    members=[
        reddit_researcher,
        academic_paper_researcher,
    ],
    instructions=[
        "You are a discussion master.",
        "You have to stop the discussion when you think the team has reached a consensus.",
    ],
    success_criteria="The team has reached a consensus.",
    enable_agentic_context=True,
    show_tool_calls=True,
    markdown=True,
    show_members_responses=True,
)

if __name__ == "__main__":
    asyncio.run(
        agent_team.print_response(
            message="Start the discussion on the topic: 'What is the best way to learn to code?'",
            stream=True,
            stream_intermediate_steps=True,
        )
    )
```

Team definition with a cutoff point.

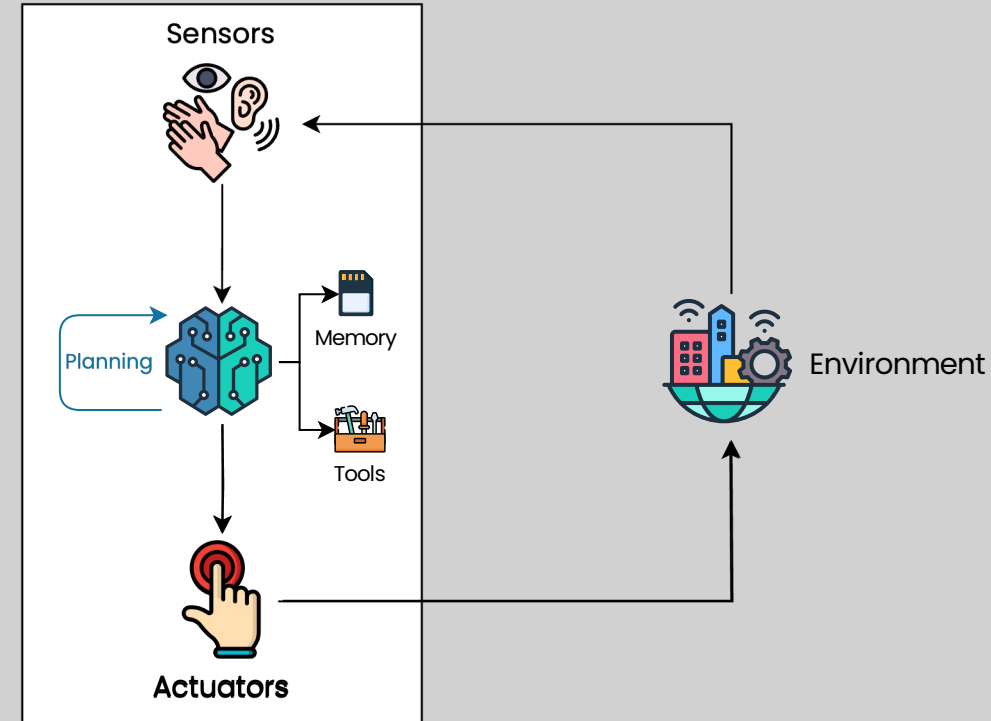
Model Context Protocol



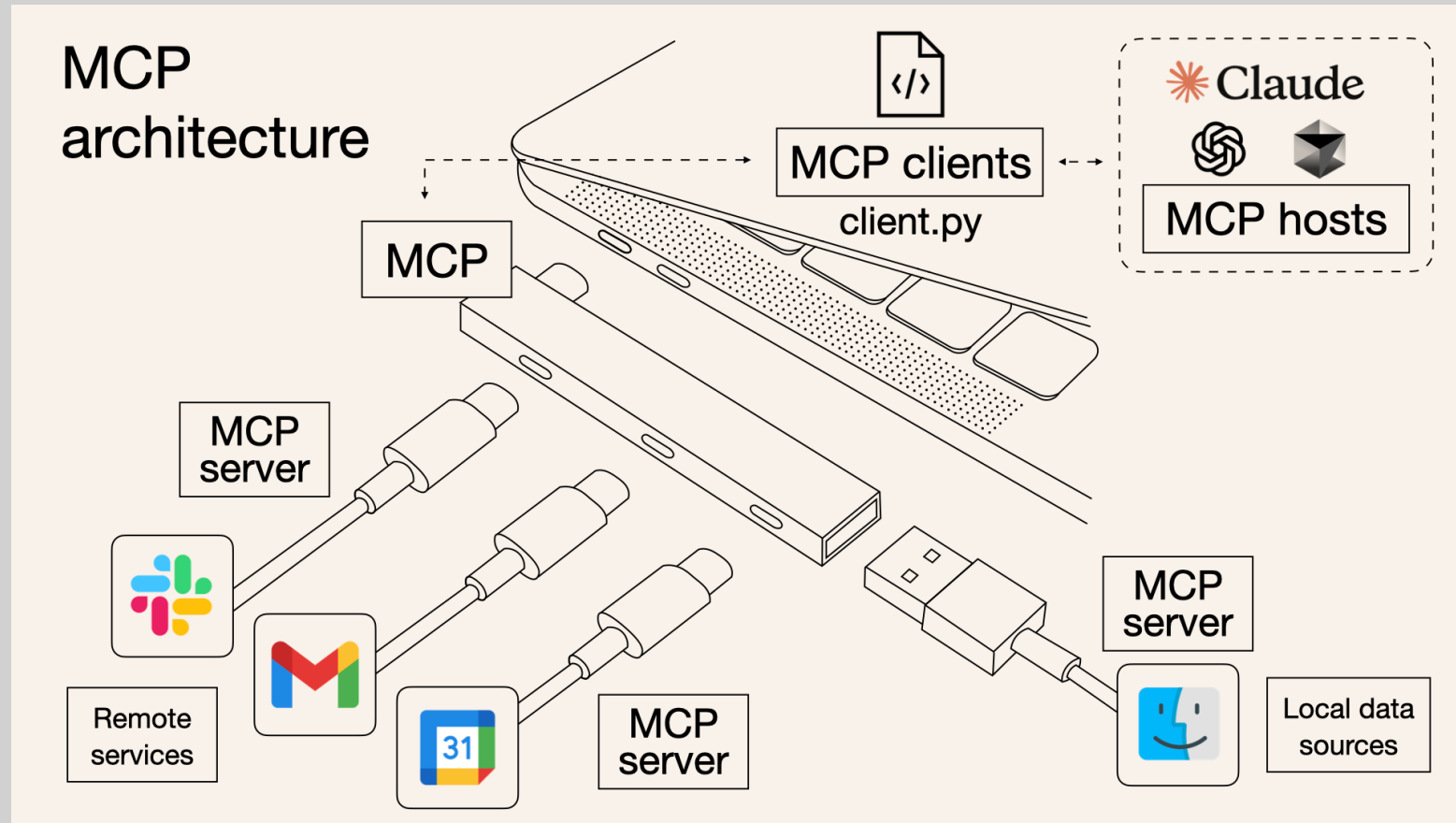
Tools

-  Manually tracked and fed to the LLM
-  Manually described (its expected schema)
-  Manually updated whenever its API changes
-  Building tools takes time and effort.
-  Each framework & app use different tool standards

Agent



Model Context Protocol

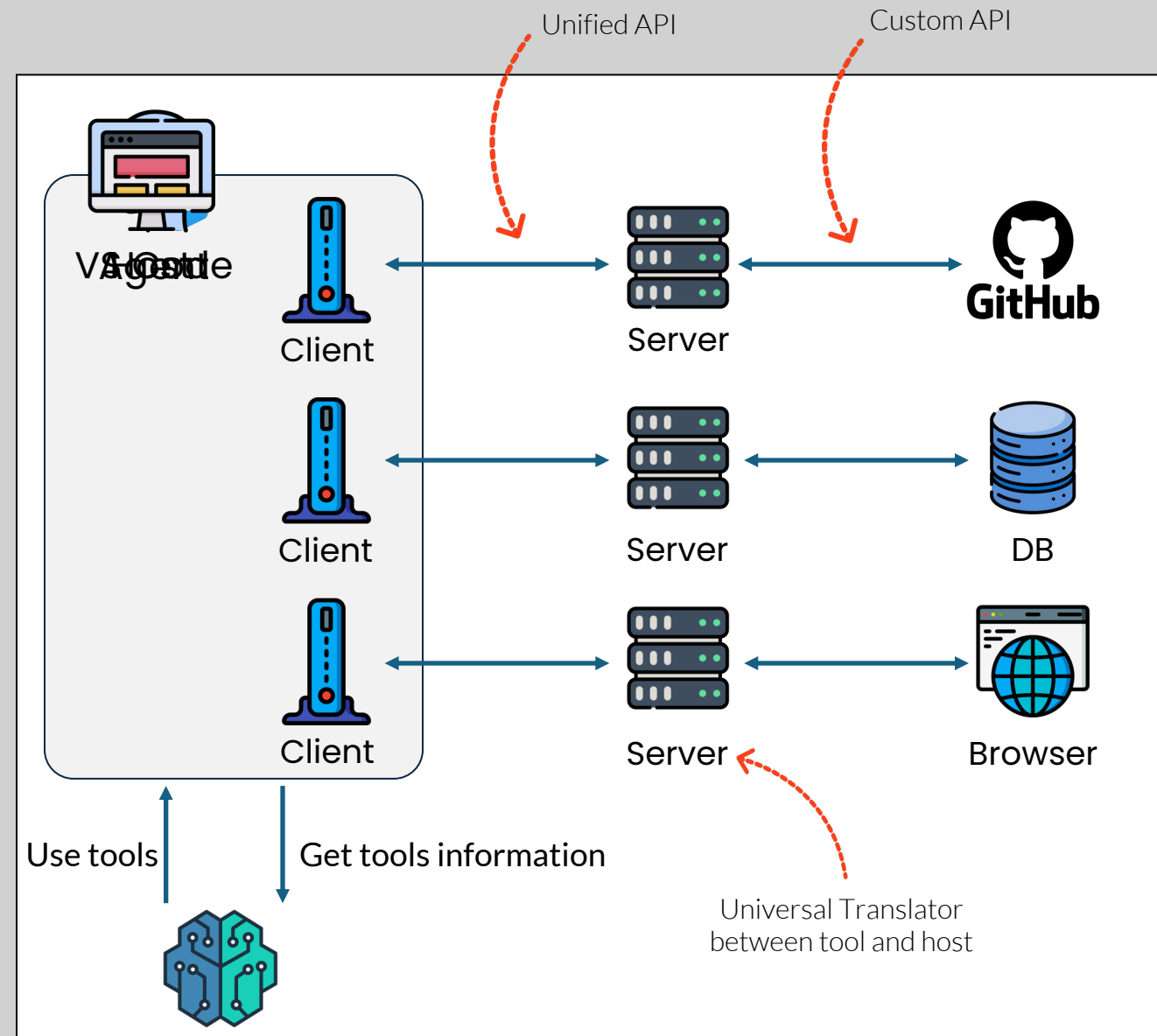


Model Context Protocol Architecture



MCP has three components:

- **Host:** LLM application that manages connections
- **Client:** Maintains connections with MCP servers
- **Server:** Provides context & tools to the LLMs



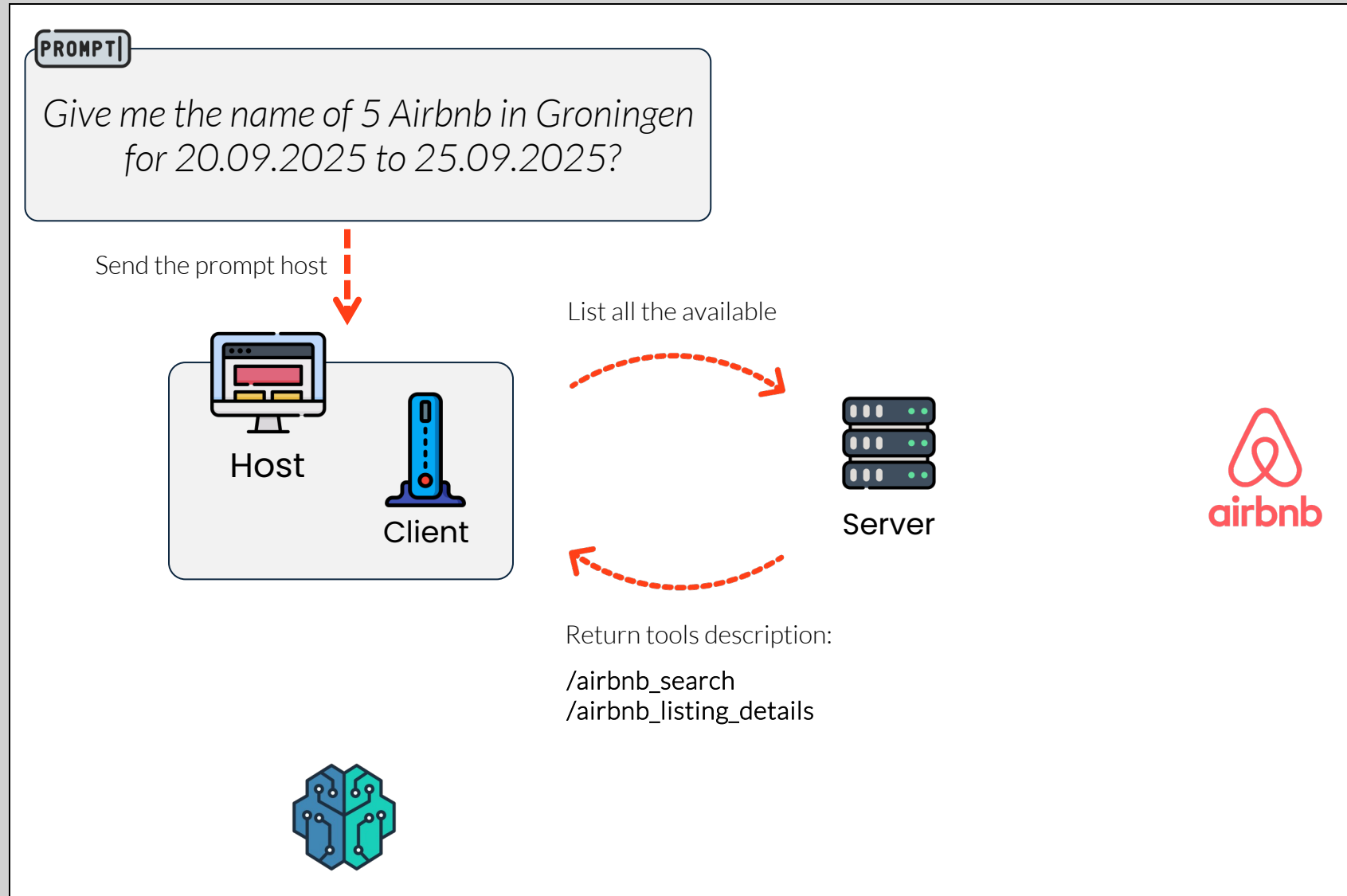
Model Context Protocol Architecture



How does MCP Work?

Stages:

 Tool Discovery

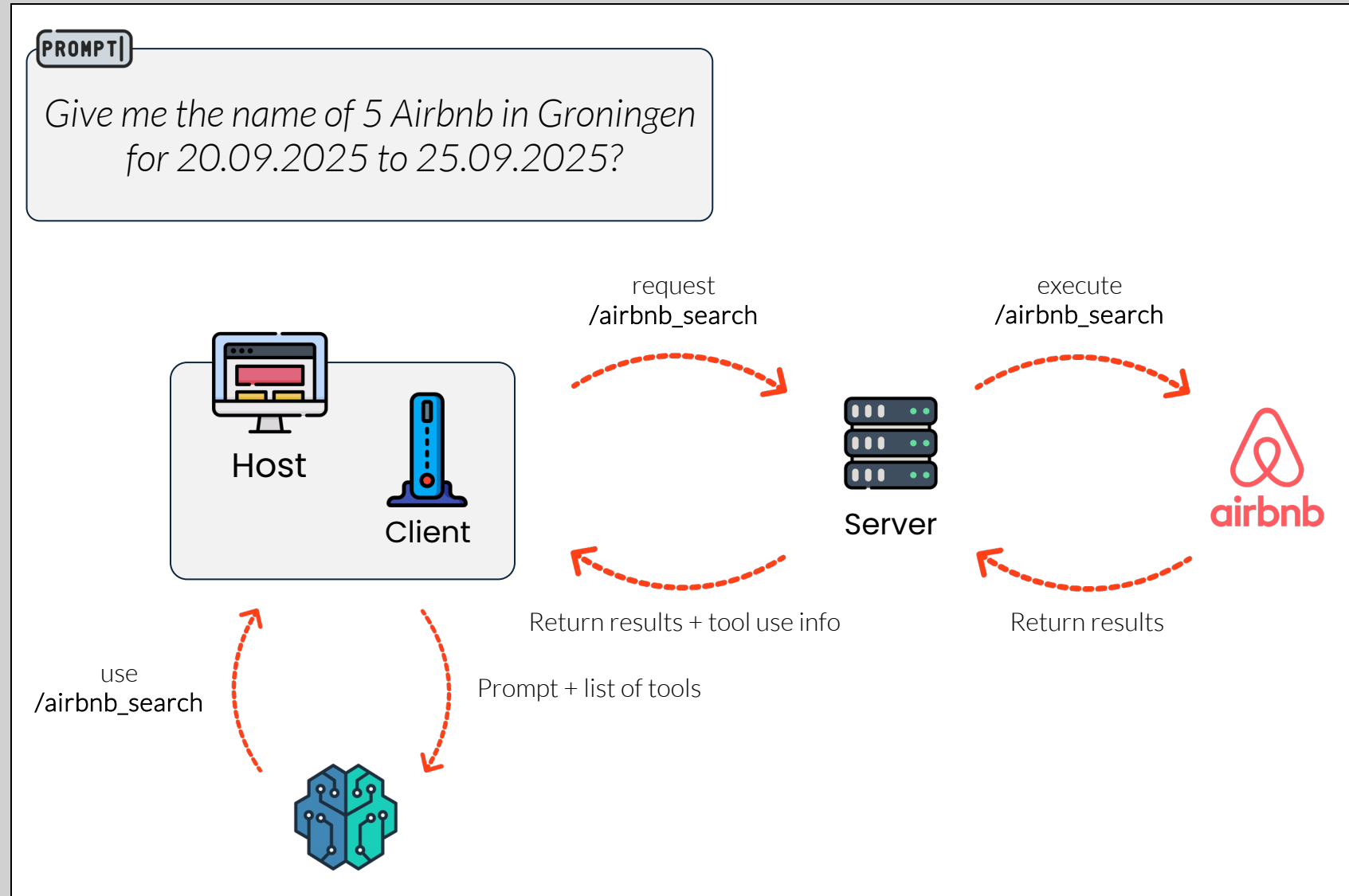


How does MCP Work?

Stages:

 Tool Discovery

 Tool Use



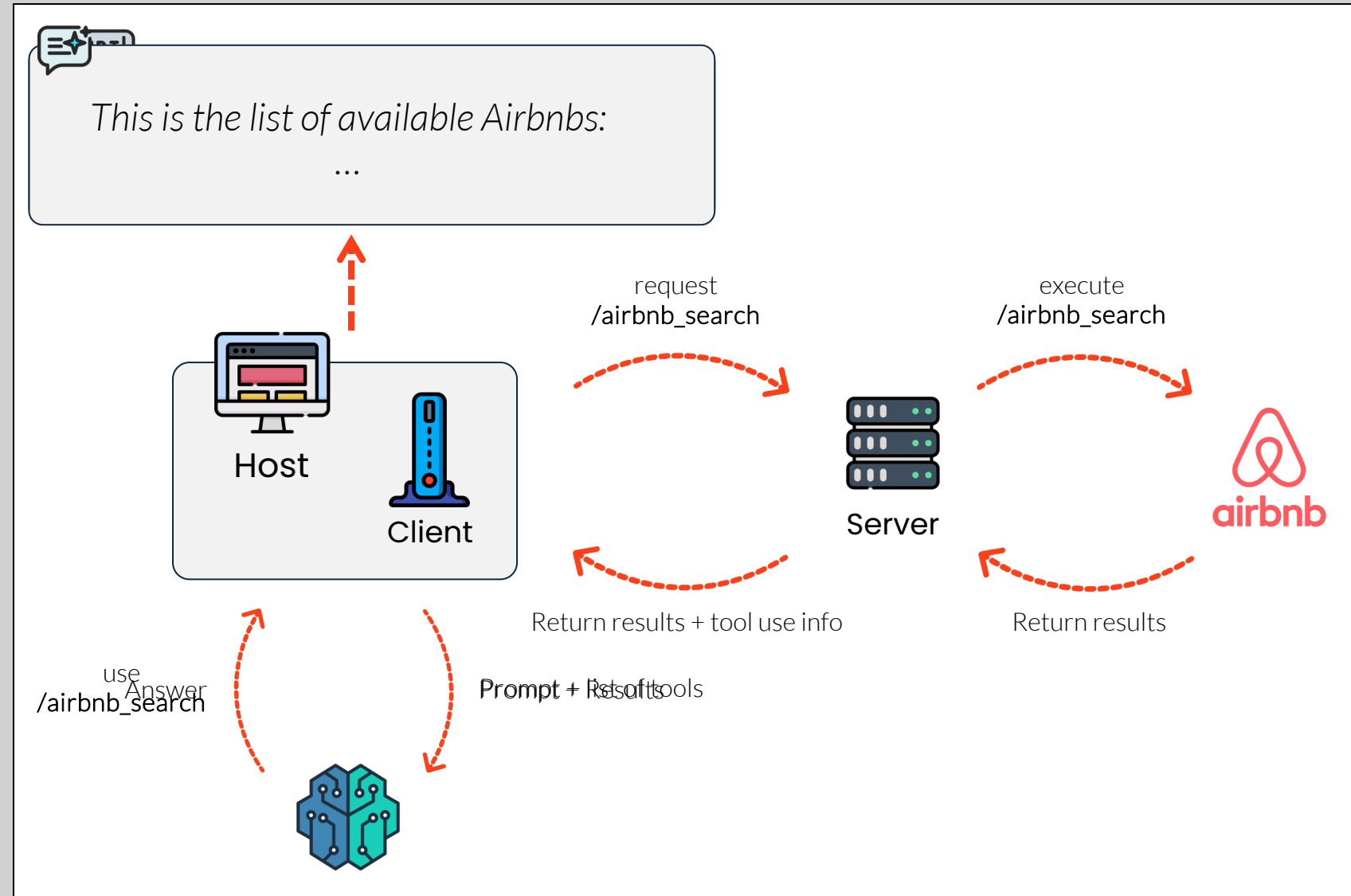
How does MCP Work?

Stages:

 Tool Discovery

 Tool Use

 Answer Generation



What can be shared through an MCP server?



Tools:

- Retrieve Data
- Interact with environment
- Interact with external systems



Resources:

- Files
- Database Records



Prompts:

- Prompts Templates
- Procedures
- Document Q&A

A surreal scene where two boxers are fighting in a ring. The boxer on the left is a Black man, and the boxer on the right is a white man. They are both wearing red boxing gloves and black trunks. In the background, a classic dark blue Mercedes-Benz 280 SE 3.5 is parked inside the boxing ring. The car's license plate reads "280 SE 3.5". The scene is lit with bright spotlights, creating a dramatic atmosphere.

And . . . Done!

AI GENERATED

Feedback and Q&A

Feedback



<https://www.menti.com/alsxdqfymhbb>

Get in touch with me

